
Contrastive Distribution Matching for Amortized Sequential Monte Carlo in Discrete Diffusion

Jaihoon Kim^{1†} Taehoon Yoon² Prin Phunyaphibarn¹
Seungjun Kim¹ Morteza Mardani³ Minhyuk Sung¹

¹KAIST ²University of Michigan ³NVIDIA

Project Page: <https://cdm-smc.github.io/>

Abstract

Discrete diffusion models have emerged as powerful frameworks for generating structured categorical data. However, efficiently sampling from reward-tilted distributions remains a fundamental challenge. While Twisted Sequential Monte Carlo (SMC) offers asymptotic exactness for this task, estimating the optimal twist function in discrete state spaces necessitates costly Monte Carlo approximations, resulting in a severe computational bottleneck at inference. To overcome this limitation, we introduce Contrastive Distribution Matching (CDM), a novel framework that amortizes the cost of SMC inference by learning a parameterized twist function via positive and negative samples. For efficient training, we reformulate the gradient estimator to leverage the closed-form forward kernels of discrete diffusion models. In practice, evaluating our learned twist function incurs less than 5% additional computational overhead compared to a single forward pass of the base model. Through extensive empirical evaluations, we demonstrate that CDM consistently outperforms existing baselines under matched wall-clock time. We validate the effectiveness and versatility of our approach across a diverse range of applications, including toxic text generation, regulatory DNA sequence design, protein designability, and diffusion large language model alignment.

1 Introduction

Diffusion models have demonstrated remarkable generative performance across a wide array of continuous domains [68, 36, 37]. Recently, their application to discrete state spaces has yielded significant breakthroughs; in language modeling, discrete diffusion models not only enable efficient few-step generation [60, 40, 56] but also achieve sample quality on par with autoregressive language models [53, 87, 88]. Furthermore, discrete diffusion has been successfully extended to scientific applications, driving advances in sequence design tasks such as regulatory DNA [43, 70] and de novo protein [84] generation.

A central application of these models is *reward alignment*. Given a scalar reward representing human preference [46, 83] or protein designability [44], the objective is to sample from a tilted target distribution that biases the pretrained prior toward higher values of the downstream reward. To sample exactly from this target distribution, the optimal proposal is formulated by tilting the pretrained base model with an *optimal twist function*.

In this work, we integrate the Sequential Monte Carlo (SMC) framework [19, 52], an asymptotically unbiased sampler, with discrete diffusion models. In the continuous domain, the SMC framework has been widely adopted for reward alignment largely due to its computational efficiency and empirical success [85, 6, 34, 73, 74, 67, 1]. This tractability stems from two key properties: one can easily

[†]Correspondence to: jh27kim@kaist.ac.kr

construct locally optimal proposals using reward gradients [11, 34], and Tweedie’s formula [20] provides a computationally efficient estimate of the clean state to approximate the twist function. In contrast, translating these successes to discrete domains presents new challenges. Since the state space is discrete, one needs to rely on Gumbel-Softmax trick [30] to approximate locally optimal proposals, which often leads to gradient bias and optimization instability [57, 47]. More importantly, the absence of Tweedie’s estimate in discrete diffusion [65] leaves Monte Carlo estimation as the standard practice for approximating the twist function [55, 43, 13], which can introduce a significant inference overhead when the downstream reward is computationally expensive (e.g., protein designability).

Motivated by this bottleneck, we propose Contrastive Distribution Matching (CDM), which learns the twist function via a contrastive learning objective to reduce the twist function evaluation to a *constant-time* operation, amortizing SMC inference. In contrast to existing regression-based methods applied to discrete diffusion that learn the twist by drawing samples from a base proposal [43, 78], CDM minimizes the forward KL divergence against the target distribution. The gradient of this objective exhibits a contrastive structure, utilizing positive and negative samples to upweight high-reward regions while downweighting suboptimal ones. Additionally, we introduce a novel training scheme that leverages the forward process of the diffusion model for efficient training. Specifically, we maintain a buffer of positive samples drawn from the approximated target distribution and apply the closed-form forward kernel, allowing a single clean sample to be reused across multiple timesteps and gradient updates.

Our experimental evaluations demonstrate that CDM consistently achieves superior scaling behavior, outperforming baselines in a diverse range of applications: toxic text generation, regulatory DNA design, protein generation, and diffusion LLM (dLLM) preference alignment. Furthermore, since CDM learns the twist function, it is agnostic to the choice of the proposal distribution. This allows it to be paired with *any* proposal distribution, including those already fine-tuned (e.g., d1 [94], DRAKES [82]), for further synergistic performance gains. Moreover, we demonstrate that the contrastive learning objective of CDM yields superior performance and more efficient training compared to the standard regression-based twist objective [78, 43].

In summary, our key contributions are as follows:

- We propose Contrastive Distribution Matching (CDM), an SMC-amortization framework for discrete diffusion that reduces the cost of applying the twist at inference time to a constant-time operation.
- We design a novel, diffusion-native training scheme that leverages the closed-form forward process, enabling efficient training that scales to expensive reward functions.
- We demonstrate the versatility of CDM across a broad range of applications, including toxic text generation, regulatory DNA design, protein generation, and dLLM alignment, consistently showing superior performance.
- We validate that CDM delivers synergistic improvements even when paired with fine-tuning-based methods, while demonstrating superior efficacy than the regression-based twist objectives employed in previous discrete diffusion models [43].

2 Preliminary: Discrete Diffusion

Let $\mathcal{V} := \{\mathbf{x} \in \{0, 1\}^V : \sum_{i=1}^V \mathbf{x}_i = 1\}$ define the space of V category one-hot vectors. We write Δ^V for the V -simplex and $\text{Cat}(\cdot; \boldsymbol{\pi})$ for the categorical distribution with probability vector $\boldsymbol{\pi} \in \Delta^V$. A prominent class of generative models for discrete state spaces is the Masked Diffusion Model (MDM) [69, 5, 72], which defines a forward corruption process terminating in a *mask* state $\mathbf{m}$.

Let $(p_t)_{t=0}^T$ denote the sequence of marginal distributions induced by this forward process. The process interpolates between the data distribution $p_0 = p_{\text{data}}$ and the prior $p_T = \mathbf{m}$ via a monotonically decreasing noise schedule $\alpha_t \in [0, 1]$:

$$p(\mathbf{x}_t \mid \mathbf{x}_0) = \text{Cat}(\mathbf{x}_t; \alpha_t \mathbf{x}_0 + (1 - \alpha_t) \mathbf{m}) \quad (1)$$

The sampling proceeds by simulating the reverse process, where the exact posterior is given by:

$$p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \begin{cases} \text{Cat}(\mathbf{x}_{t-1}; \mathbf{x}_t) & \text{if } \mathbf{x}_t \neq \mathbf{m}, \\ \text{Cat}\left(\mathbf{x}_{t-1}; \frac{(1 - \alpha_{t-1})\mathbf{m} + (\alpha_{t-1} - \alpha_t)\mathbf{x}_0}{1 - \alpha_t}\right) & \text{if } \mathbf{x}_t = \mathbf{m}. \end{cases} \quad (2)$$

Since the clean data $\mathbf{x}_0$ is unknown during sampling, it is approximated by a denoising neural network $\mathbf{x}_\theta(\mathbf{x}_t) \in \Delta^V$. Substituting this prediction into the posterior Eq. (2) yields the parameterized reverse transition kernel $p^{\text{base}}(\mathbf{x}_{t-1} | \mathbf{x}_t) = p(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_\theta(\mathbf{x}_t))$. The resulting reverse chain induces a trajectory distribution $p_{0:T}^{\text{base}}(\mathbf{x}_{0:T})$ with time marginals $p_t^{\text{base}}(\mathbf{x}_t)$. The model parameters θ are optimized by minimizing a weighted cross entropy loss which is equivalent to the negative ELBO in the continuous-time limit. We refer to previous works [5, 72] for the detailed derivations.

3 Twisted Sequential Monte Carlo for Reward Alignment

3.1 KL Regularized Reward Alignment

Let $r : \mathcal{V}^N \rightarrow \mathbb{R}$ be a reward function that maps a fully denoised sequence of length N to a scalar (e.g., human preference score or protein validity). Given this reward function and a pretrained model distribution $p_{0:T}^{\text{base}}$ parameterized by an MDM, our objective is to maximize the expected reward while penalizing deviations from the base model [63, 31]:

$$p_{0:T}^* = \arg \max_{p_{0:T}} \mathbb{E}_{p_{0:T}}[r(\mathbf{x}_0)] - \beta \mathcal{D}_{\text{KL}}(p_{0:T} \| p_{0:T}^{\text{base}}), \quad (3)$$

where $\beta > 0$ is a hyperparameter controlling the strength of the KL regularization.

The intermediate target distribution $p_t^*(\mathbf{x}_t)$ that maximizes the objective in Eq. (3) can be derived in a closed form:

$$p_t^*(\mathbf{x}_t) = \frac{1}{Z_t} p_t^{\text{base}}(\mathbf{x}_t) \psi_t^*(\mathbf{x}_t) \quad \psi_t^*(\mathbf{x}_t) = \exp\left(\frac{V_t(\mathbf{x}_t)}{\beta}\right), \quad (4)$$

where $\psi_t^*(\mathbf{x}_t)$ is the *optimal twist function* that modulates the base distribution to match the target, given as the exponentiated *optimal value function* $V_t(\mathbf{x}_t)$. A classic result states that the optimal value function can be expressed using the base model posterior [32, 77, 17]:

$$V_t(\mathbf{x}_t) = \beta \log \mathbb{E}_{p^{\text{base}}(\mathbf{x}_0 | \mathbf{x}_t)} \left[\exp\left(\frac{r(\mathbf{x}_0)}{\beta}\right) \middle| \mathbf{x}_t \right]. \quad (5)$$

Sampling from the target distribution in Eq. (4) can be performed using Twisted Sequential Monte Carlo, whose importance weights require evaluating the optimal twist function. The central challenge is therefore how to estimate this twist function efficiently. In continuous diffusion, this quantity is often approximated using Tweedie’s formula [20], which provides the posterior mean of the clean sample. This yields the plug-in estimate $r(\mathbb{E}_{p^{\text{base}}(\mathbf{x}_0 | \mathbf{x}_t)}[\mathbf{x}_0])$, which has been shown to be effective in practice [11, 76]. However, discrete diffusion lacks an analogous relation [65], leaving costly Monte Carlo estimation as the standard practice [55, 13]. To address this challenge in discrete diffusion, we propose learning the twist function in advance to amortize this inference cost.

3.2 Twisted Sequential Monte Carlo

We consider the problem of sampling from the target distribution p_t^* presented in Eq. (4). Given a proposal distribution $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ and the unnormalized trajectory-level target $\tilde{p}_{t:T}(\mathbf{x}_{t:T}) := p_{t:T}^{\text{base}}(\mathbf{x}_{t:T}) \psi_t^*(\mathbf{x}_t)$, Sequential Monte Carlo (SMC) [8] interleaves sequential importance sampling with particle resampling to approximate the target distribution [14, 19]. Specifically, the unnormalized importance weight is calculated at each step as³:

$$w_{t-1} = \frac{\tilde{p}_{t-1:T}(\mathbf{x}_{t-1:T})}{\tilde{p}_{t:T}(\mathbf{x}_{t:T}) q(\mathbf{x}_{t-1} | \mathbf{x}_t)} = \frac{\psi_{t-1}^*(\mathbf{x}_{t-1}) p^{\text{base}}(\mathbf{x}_{t-1} | \mathbf{x}_t)}{\psi_t^*(\mathbf{x}_t) q(\mathbf{x}_{t-1} | \mathbf{x}_t)}, \quad (6)$$

where we adopt the Markov assumption on the target trajectory [34, 55]. Given K particles $\{\mathbf{x}_t^{(k)}\}_{k=1}^K$, the normalized weights $\{\tilde{w}_t^{(k)}\}_{k=1}^K$ yield a target approximation $p_t^* \approx \sum_{k=1}^K \tilde{w}_t^{(k)} \delta_{\mathbf{x}_t^{(k)}}$.

The optimal proposal distribution has a closed form expression $q^*(\mathbf{x}_{t-1} | \mathbf{x}_t) \propto p^{\text{base}}(\mathbf{x}_{t-1} | \mathbf{x}_t) \psi_{t-1}^*(\mathbf{x}_{t-1})$, which minimizes the variance of the importance weights in Eq. (6). However, this optimal proposal is generally intractable, since computing its normalizing constant requires evaluating the twist function over all possible next states.

³We assume resampling at every step. See Appendix A.1 for a detailed discussion of SMC.

In the cases when the reward is differentiable, one can approximate the optimal proposal via a first-order Taylor expansion [55]:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t) \propto p^{\text{base}}(\mathbf{x}_{t-1}|\mathbf{x}_t) \exp(\mathbf{x}_{t-1}^\top \nabla_{\mathbf{x}} \log \psi_t^*(\mathbf{x}_t)). \quad (\text{Grad})$$

This is the discrete counterpart to gradient-based guidance, an approach that has proven highly effective in the continuous domain [11, 90, 3]. However, this method exhibits two key limitations. First, since discrete state spaces are inherently non-differentiable, computing the gradients relies on the Gumbel-Softmax trick [30], which often suffer from gradient bias and optimization instability [57]. Second, and more importantly, this approach is fundamentally incompatible with non-differentiable objectives (e.g., API-based rewards).

Proposal Distributions. We consider two gradient-free alternatives that sidestep the differentiability requirement of Eq. (Grad). We can either use the pretrained base transition kernel directly or fine-tune the pretrained model [82, 61, 26, 94] and use the resulting reward-aware proposal $p^{\text{FT}}(\mathbf{x}_{t-1}|\mathbf{x}_t)$. Each choice of transition kernel will result in the following importance weights, respectively:

$$w_{t-1} = \frac{\psi_{t-1}^*(x_{t-1})}{\psi_t^*(x_t)} \quad (\text{SMC}) \quad w_{t-1} = \frac{\psi_{t-1}^*(x_{t-1}) p^{\text{base}}(x_{t-1} | x_t)}{\psi_t^*(x_t) p^{\text{FT}}(x_{t-1} | x_t)} \quad (\text{FT+SMC})$$

In both cases, the importance weight depends on the twist ratio ψ_{t-1}^*/ψ_t^* , which plays a key role in the accuracy of the target approximation.

3.2.1 Motivation: Monte Carlo Twist Function Estimation

As discussed in Sec. 3.1, while the twist function can be cheaply estimated in continuous diffusion, the discrete case relies on Monte Carlo estimation. This approach draws M clean samples from the base model posterior and averages the exponentiated rewards [55, 43, 13]:

$$\psi_t^*(\mathbf{x}_t) \approx \frac{1}{M} \sum_{m=1}^M \exp\left(\frac{r(\hat{\mathbf{x}}_0^{(m)})}{\beta}\right), \quad \{\hat{\mathbf{x}}_0^{(m)}\}_{m=1}^M \sim p^{\text{base}}(\mathbf{x}_0|\mathbf{x}_t). \quad (7)$$

Although this estimate becomes exact as $M \rightarrow \infty$, scaling M incurs significant inference overhead.

Fig. 1 illustrates the reward and wall-clock time of SMC in the protein generation task as M increases. While a larger M provides a more accurate estimate of the twist function, thereby leading to consistent improvements in reward alignment for both base and fine-tuned proposals, it also increases inference time proportionally. This overhead becomes prohibitive when the reward evaluation is computationally expensive. To address this, we propose a contrastive learning framework that amortizes the twist computation by training a network to directly predict the optimal twist function ψ_t^* in a single forward pass. This reduces the twist evaluation to a *constant-time* operation and remains applicable regardless of the chosen proposal for further improvements.

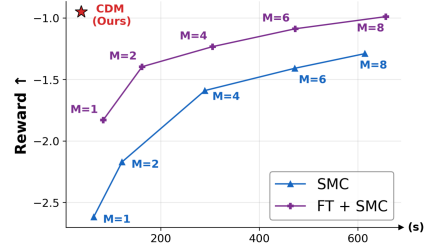


Figure 1: **Reward vs. Wall-Clock Time with Varying M .** Increasing M yields a more accurate twist estimate, improving SMC performance, but incurs a substantial computational cost. CDM show superior scalability by amortizing this cost.

4 Amortized SMC with a Learned Twist Function

In this section, we first review the standard regression-based approach and then introduce a contrastive learning objective designed for MDMs.

Regression Objective. Let $\psi_t^\phi(\mathbf{x}_t)$ denote a parameterized neural network. The model is trained via direct regression by minimizing the Mean Squared Error between the network prediction and the optimal twist function in Eq. (4):

$$\min_{\phi} \mathbb{E}_{t, \mathbf{x}_t \sim p_t^{\text{base}}} \left[\left(\psi_t^\phi(\mathbf{x}_t) - \mathbb{E}_{p^{\text{base}}(\mathbf{x}_0|\mathbf{x}_t)} \left[\exp\left(\frac{r(\mathbf{x}_0)}{\beta}\right) \mid \mathbf{x}_t \right] \right)^2 \right]. \quad (\text{Soft Value})$$

In practice, the optimal twist target is approximated via a Monte Carlo estimate with M samples as in Eq. (7). This regression target is a special case of soft Q-learning from the RL literature [42, 35]

with no intermediate reward, and has been widely adopted in prior work on diffusion models [43, 78]. For simplicity, we refer to twist-learning methods trained with this objective as *Soft Value*.

While straightforward, this objective trains the twist function on samples drawn from the reward-agnostic base distribution. Therefore, the model is trained on a distribution that does not necessarily reflect the target distribution at inference, resulting in a train-test distributional mismatch. As a result, the learned twist target can be inaccurate in the regions most relevant for target sampling, leading to suboptimal performance. This motivates a distribution-level matching objective, which naturally yields a contrastive learning formulation based on positive and negative samples.

4.1 CDM: Contrastive Distribution Matching

The core of our formulation lies in aligning the distribution induced by the twist function with the optimal target. Drawing inspiration from recent work on autoregressive language models [95], we utilize the forward KL divergence. Specifically, let $p_t^\phi(\mathbf{x}_t)$ denote an intermediate distribution where the base distribution is modulated by the parameterized twist function $\psi_t^\phi(\mathbf{x}_t)$:

$$p_t^\phi(\mathbf{x}_t) = \frac{1}{Z_t^\phi} p_t^{\text{base}}(\mathbf{x}_t) \psi_t^\phi(\mathbf{x}_t), \quad Z_t^\phi = \sum_{\mathbf{x}_t} p_t^{\text{base}}(\mathbf{x}_t) \psi_t^\phi(\mathbf{x}_t). \quad (8)$$

To align p_t^ϕ with the optimal target p_t^* at each timestep, we minimize the following time-averaged forward KL objective:

$$\mathcal{L}_{\text{CDM}}(\phi) = \mathbb{E}_t \left[\mathcal{D}_{\text{KL}} \left(p_t^*(\mathbf{x}_t) \parallel p_t^\phi(\mathbf{x}_t) \right) \right], \quad (9)$$

which we refer to as Contrastive Distribution Matching, CDM. To understand the contrastive behavior of this objective, we analyze the gradient of the loss function with respect to the parameters ϕ :

$$-\nabla_\phi \mathcal{L}_{\text{CDM}}(\phi) = \mathbb{E}_t \left[\underbrace{\mathbb{E}_{p_t^*(\mathbf{x}_t)} \left[\nabla_\phi \log \psi_t^\phi(\mathbf{x}_t) \right]}_{\text{Positive Term}} - \underbrace{\mathbb{E}_{p_t^\phi(\mathbf{x}_t)} \left[\nabla_\phi \log \psi_t^\phi(\mathbf{x}_t) \right]}_{\text{Negative Term}} \right], \quad (10)$$

with the derivation deferred to Appendix A.2. Note that the gradient exhibits a contrastive structure: the positive term increases $\log \psi_t^\phi$ on samples drawn from the target distribution p_t^* , while the negative term decreases it on samples drawn from the current approximation p_t^ϕ . The positive term mitigates the train-test distributional mismatch described previously, whereas the negative term calibrates the learned distribution by suppressing suboptimal samples. Leveraging both positive and negative samples is known to yield more effective training, as also observed in previous works [96, 16, 7, 99]. Next, we explain how we effectively adapt it to diffusion models with a sampling scheme designed for efficient training.

4.2 Efficient Twist Training

To evaluate the contrastive gradient, we first need to address the problem of sampling from the optimal target distribution p_t^* . Since direct sampling from the target distribution is intractable, one can approximate sampling from p_t^* via importance sampling (IS) or SMC, using the pretrained base model as a proposal. The optimal twist function appearing in the importance weights is then estimated with the Monte Carlo approximation in Eq. (7). While sampling from p_t^* may appear circular, these samples serve as training targets that amortize the cost of all subsequent inference-time sampling.

Under the IS framework, we draw trajectories from the pretrained base model and compute the importance weights only at timestep t , making this approach computationally efficient. However, in practice, we observe that it suffers from high variance when drawing positive samples.

SMC mitigates this variance by interleaving intermediate reweighting with particle resampling, yielding positive samples that are better aligned with the reward-tilted target. However, this improved sample quality comes at the cost of sequential weight computations, which require repeated queries to the reward model. This trade-off becomes especially pronounced when the reward model is expensive, motivating a more efficient training scheme.

Forward-Based Gradient Estimation. A fundamental limitation of Eq. (10) is that it allows only a *single* gradient update per positive sample. This sample inefficiency creates a bottleneck, particularly

severe when reward evaluation is computationally expensive. We address this by exploiting a diffusion-specific property of the target marginals. Rather than sampling independently from each intermediate target p_t^* , we first obtain clean positive samples from p_0^* . By leveraging the closed-form diffusion forward kernel, we can then draw multiple positive samples at any intermediate timestep at negligible cost. Specifically, the intermediate target decomposes as $p_t^*(\mathbf{x}_t) = \sum_{\mathbf{x}_0} p_0^*(\mathbf{x}_0) p^{\text{base}}(\mathbf{x}_t | \mathbf{x}_0)$ (Appendix A.3), which is a structural advantage unique to diffusion frameworks and unavailable to standard autoregressive models [95, 86, 51]. Leveraging this decomposition, we reformulate the gradient in Eq. (10) as:

$$-\nabla_{\phi} \mathcal{L}_{\text{CDM}}(\phi) = \mathbb{E}_t \left[\mathbb{E}_{p_0^*(\mathbf{x}_0)} \mathbb{E}_{p^{\text{base}}(\mathbf{x}_t | \mathbf{x}_0)} \left[\nabla_{\phi} \log \psi_t^{\phi}(\mathbf{x}_t) \right] - \mathbb{E}_{p_t^{\phi}(\mathbf{x}_t)} \left[\nabla_{\phi} \log \psi_t^{\phi}(\mathbf{x}_t) \right] \right], \quad (11)$$

yielding an unbiased gradient estimator. This forward-based formulation enables an efficient buffer-based training scheme in which we maintain a buffer $\mathcal{B}$ of clean positive samples and repeatedly apply the forward kernel across timesteps to obtain multiple gradient updates from each sample [98, 28], thereby effectively reducing the cost of reward evaluations throughout training.

Negative Sampling. One can utilize the IS/SMC framework to sample from p_t^{ϕ} by replacing the optimal twist function in Eq. (6) with our parameterized twist, ψ_t^{ϕ} . Note that unlike the positive sampling case, p_t^{ϕ} does not admit a forward-kernel decomposition under the base process. In practice, we find that for negative sampling, IS achieves effective performance while being more computationally efficient than SMC. This efficiency ensures that the overall negative sampling procedure remains highly scalable.

Beyond the choice of negative sampler, we observe that purely online training of Eq. (11) exhibits optimization instability. To mitigate this, we adopt the soft target update from the RL literature [9, 96] and maintain an exponential moving average (EMA) of the twist parameters. The detailed training algorithm for CDM is presented in Appendix B.

4.3 Efficient Twist Parameterization

An efficient parameterization of the twist function is critical, as we aim to amortize the expensive computational cost of Monte Carlo estimation via a single forward pass of our learned model, ψ_t^{ϕ} . A straightforward implementation would train a separate network for ψ_t^{ϕ} from scratch, but this introduces non-negligible inference overhead, since computing the importance weights in Eq. (6) requires evaluating ψ_t^{ϕ} for every particle at each denoising step.

To minimize this cost, we parameterize ψ_t^{ϕ} as a lightweight scalar head attached to the final feature layer of the pretrained model, alongside the existing logit head (Fig. 6). Thus, once the backbone features are computed, the model can produce both the logits and the twist estimate in a single forward pass through their respective heads. This parameterization adds negligible computational overhead, around 5% of the backbone forward-pass time and as little as 0.5%. As a result, sampling with the learned twist is essentially as fast as standard sampling from the base model, and the number of particles can be scaled well beyond that of the SMC baseline in Eq. (SMC). This parameterization contrasts favorably with prior approaches that train isolated value networks from scratch [43, 78], which not only incur non-negligible inference-time overhead but also fail to leverage the rich representations already learned by the diffusion backbone. We detail the twist-head architecture and parameterization in Appendix. C.

5 Related Work

Proposal Fine-Tuning. In the continuous domain, aligning diffusion models with downstream rewards typically involves either direct backpropagation across the sampling trajectory [12, 61] or reformulating the denoising steps as a Markov decision process to enable reinforcement learning [4, 21, 81]. While highly effective, adapting these methods to discrete state spaces necessitates specialized adaptations. d1 [94] employs a mean-field approximation to utilize the GRPO objective [71], whereas DRAKES [82] enables direct backpropagation through a Gumbel-Softmax relaxation [30]. Other approaches incorporate importance sampling [91, 98, 99] to estimate likelihood ratios, or compute adjoint states [75]. Crucially, our amortized SMC framework is complementary to this body of work. We emphasize that *any* fine-tuned model can be integrated as a proposal distribution within our framework to achieve further performance scaling.

Inference-Time Scaling. Inference-time scaling offers a training-free alternative, with gradient-based guidance serving as a notable example in both continuous [11] and discrete spaces [54]. However, computing gradient guidance in discrete state spaces degrades when the linearity assumption is violated [92, 25], and, more importantly, is fundamentally limited when the reward is not differentiable. Instead, search-based methods attempt to identify high-reward trajectories by allocating more compute for exploration during sampling process [33, 64, 43, 50]. Other approaches rely on MCMC techniques, such as Gibbs sampling [10] or Metropolis-Hastings [58], to iteratively draw samples from the target distribution. However, both search- and MCMC-based methods are time-consuming and computationally heavy at inference. Our method addresses this by amortizing the inference cost through learning the twist function.

Sequential Monte Carlo for Generative Modeling. SMC [14, 18] is a widely used framework for probabilistic inference. Combined with continuous diffusion models, SMC is widely utilized for solving inverse problems [85, 6] and sampling from reward-tilted distributions [67, 74, 34, 89]. This paradigm has been extended to discrete diffusion [41, 27, 73, 13]. While some works consider fine-tuning the proposal distribution to amortize sampling costs [55, 29], the cost of inference remains a bottleneck for expensive twist function estimation. In this work, we focus on amortizing this computational cost by learning the twist function, an approach that is orthogonal to proposal fine-tuning and can be combined with it for synergistic improvements.

Learning the twist functions has been studied in the context of autoregressive language modeling [86, 51, 39] and planning [45, 59]. A notable example is soft Q-learning from RL literature [42, 66], which has recently been extended to the diffusion setting [43, 78]. Alternatively, Zhao et al. [95] proposed a contrastive learning objective for autoregressive language models, though their formulation is specifically designed for sequential text generation. We propose a contrastive objective for diffusion models that exploits the closed-form forward kernel of the diffusion process, yielding an efficient training scheme that scales to settings with expensive reward computation.

6 Experiments

We evaluate CDM across a diverse set of discrete diffusion reward alignment tasks. Sec. 6.1 benchmarks our method against inference-time baselines on text generation and biological sequence design. Sec. 6.2 integrates CDM with fine-tuned proposals to evaluate their synergistic performance. Finally, Sec. 6.3 validates our contrastive objective by comparing its training convergence against Soft Value. We provide implementation details and ablation studies in Appendix. C-D.

6.1 Applications

Baselines. We compare CDM against a representative set of scaling baselines. Best-of- N (BoN) draws independent samples from the base model and selects the highest-reward candidate. We evaluate the performance of SMC under two settings, where the twist function is approximated as defined in Eq. (7) with $M \in \{1, 4\}$. Soft Value [43, 78] denotes the case which learns the twist function using the regression objective defined in Eq. (Soft Value). For tasks with differentiable rewards, we additionally compare against SMC+Grad [55], which approximates the optimal proposal in via a first-order Taylor expansion Eq. (Grad). Lastly, we include base model to serve as a reference.

For all applications, we match training wall-clock time for methods that require optimization (e.g., Soft Value and CDM), and report scaling results against wall-clock time. In addition to the given reward used for scaling, we report a heldout reward that is not observed during scaling. We detail the experimental setup for each application below, and qualitative results are provided in Appendix F.

6.1.1 Toxic Text Generation

Experiment Setup. We consider the task of toxic text generation for evaluating our framework. For the base model, we adopt a DiT-parameterized MDM [69] pretrained on OpenWebText [23]. Following previous work [55], we use 15 prompts for evaluation with metrics averaged over 20 independent runs. The reward is the toxicity score predicted by a pretrained classifier [48]. We additionally report a heldout reward computed from another classifier which was trained on multilingual dataset [15].

Results. Fig. 2 (a-b) show the scaling results on toxic text generation for the given and heldout reward, respectively. BoN scales inefficiently in this setting, consistent with observations in previous

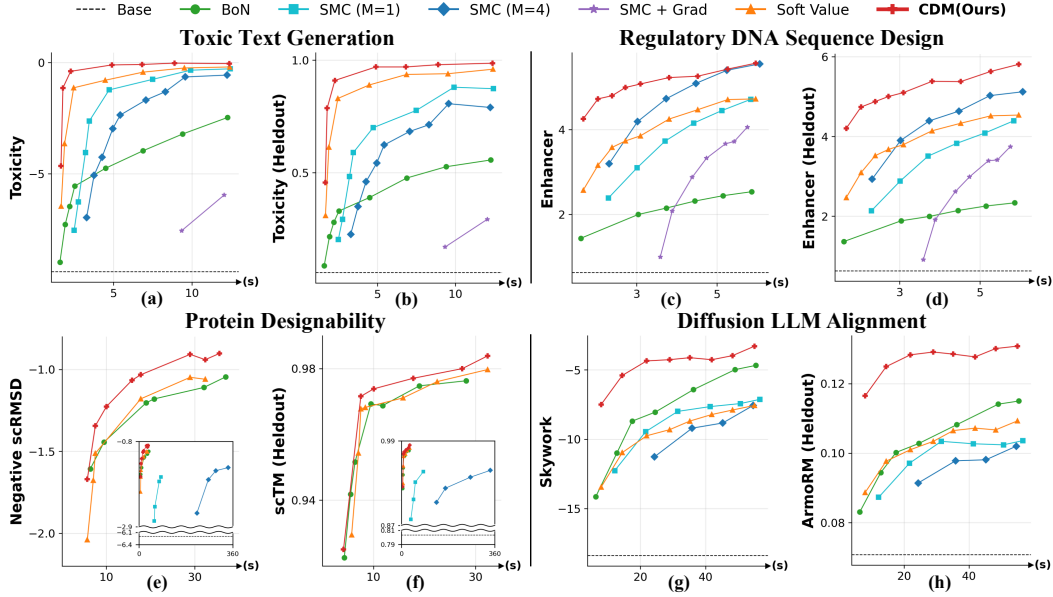


Figure 2: **Scaling Results.** We present scaling results for toxic text generation (a-b), regulatory DNA sequence design (c-d), protein designability (e-f), and diffusion LLM alignment (g-h). For each case, we plot the given reward and a heldout reward not seen during training against inference wall-clock time. In all cases, CDM establishes a new Pareto front by consistently outperforming all baselines.

works [55, 73]. SMC+Grad incurs additional runtime from gradient backpropagation through the reward, lagging behind the other baselines in compute-matched comparisons. SMC exhibits more favorable scaling but is bottlenecked by the per-step Monte Carlo twist estimation, falling short of Soft Value [43] which amortizes this cost via a learned twist head. CDM achieves the best scaling performance, outperforming all baselines across both given and heldout reward.

6.1.2 Regulatory DNA Sequence Design

Experiment Setup. Following prior works [43, 82], we train a CNN-based MDM [69] on an enhancer activity dataset consisting of 700,000 DNA sequences [24]. The given reward is provided by an Enformer model [2] trained to predict the enhancer activity in the HepG2 cell line. For the heldout reward, we utilize a separate Enformer model independently trained on the validation set.

Results. Fig. 2(c-d) show scaling performance for DNA sequence design across both given and heldout rewards. Similar to the toxicity task, BoN and SMC+Grad [55] lag behind other methods as computational budget increases. CDM maintains superior scaling, with only the SMC $M = 4$ providing comparable results on the given reward, though it remains inferior on the heldout reward.

6.1.3 Protein Designability

Experiment Setup. We consider protein generation using DPLM-2 [84] which co-generates amino acid sequences and structures, both of which are represented as discrete tokens. For the generated protein to be designable, the generated amino acid sequence should fold into the generated structure. This is quantified via a self-consistency metric by using a folding model [44] to predict the structure of the generated sequence and computing a distance measure between the predicted and generated structure. The given reward is the self-consistency RMSD (*scRMSD*), which uses RMSD as the distance metric. This reward is expensive to evaluate due to the folding model predictions, representing a regime where reward computation is the dominant bottleneck. As a heldout reward, we report *scTM*, which substitutes TM-score [93] in place of RMSD.

Results. Fig. 2(e-f) show the scaling results on the valid protein generation task for the given and heldout rewards, respectively. We present SMC in a separate inset, as its scaling is too slow to fit on the shared axis. This highlights a critical limitation of applying SMC to discrete diffusion as shown in Sec. 3.2.1. When rewards are expensive to compute, evaluating importance weights (Eq. (6))

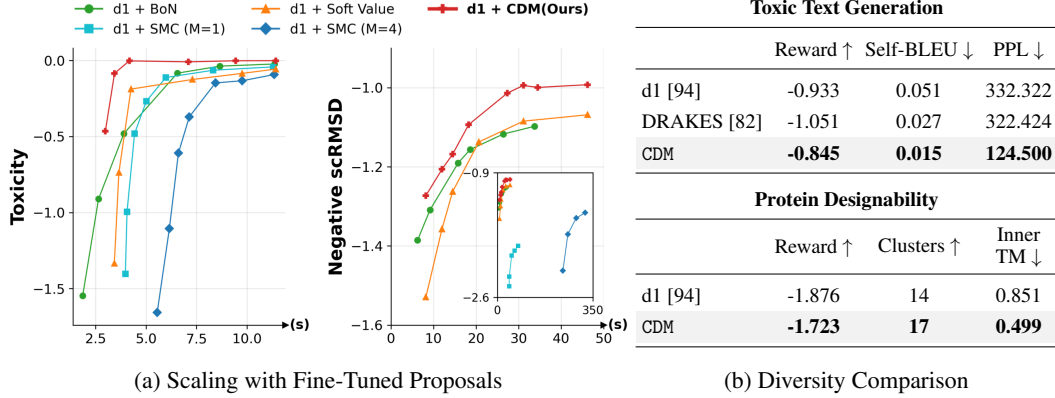


Figure 3: **Compatibility with Fine-Tuned Proposals.** (Left) Applying CDM on top of fine-tuned models improves performance for both toxic text and protein generation. (Right) CDM mitigates mode collapse commonly observed in fine-tuned models while achieving comparable rewards.

introduces a severe computational bottleneck at inference. We observe that BoN scales comparably to Soft Value [43], which we assume is because the base model already assigns non-negligible density to high-reward samples. Notably, even in this setting with expensive reward evaluations, CDM achieves superior scaling compared to both BoN and Soft Value across the given and heldout rewards.

6.1.4 Diffusion Large Language Model Alignment

Experiment Setup. We adopt LLaDA-8B-Instruct [53] as the base diffusion large language model (dLLM). We train Soft Value [43] and CDM on the training split of RewardBench [38], and evaluate on the validation split. The given reward is the preference score from the Skywork Llama-3.1-8B model [46], which is evaluated via non-differentiable API calls and represents another computationally expensive reward scenario. The heldout reward is computed using the ArmoRM reward model [83].

Results. Fig. 2(g–h) shows the scaling results for the dLLM alignment task across given and heldout rewards. As shown in the protein designability experiments in Sec. 6.1.3, SMC scales poorly due to the high cost of reward computation. While BoN demonstrates favorable scaling, it is consistently outperformed by CDM. This superiority is even more pronounced on the heldout reward, confirming the scalability of CDM to large-scale models.

6.2 Comparison with Fine-Tuned Proposals

We consider two proposal fine-tuning-based methods: d1 [94], which adapts GRPO [26] to the discrete diffusion setting, and DRAKES [82], which directly backpropagates the reward through the sampling process. Due to page limit, the scaling plot for DRAKES is presented in Appendix E.

Compatibility with Fine-Tuned Proposals. We evaluate the compatibility of CDM with fine-tuned proposals $p^{\text{FT}}(\mathbf{x}_{t-1}|\mathbf{x}_t)$ by pairing them with the scaling methods from Sec. 6.1 to achieve further performance gains, while reusing the same twist head trained independently of the fine-tuned models. Fig. 3a presents the scaling behavior of the fine-tuned proposal on the toxic text and protein generation tasks, plotted against wall-clock time. While all baselines gain from scaling, CDM combined with fine-tuned models outperforms them in both tasks, highlighting its compatibility with fine-tuned proposals for synergistic improvements.

Mode Collapse of Fine-Tuned Proposals. Fine-tuned methods often exhibit mode collapse behaviors [22, 34] which can be particularly pronounced in applications that require high diversity, such as text or protein generation. In this experiment, we analyze the diversity of fine-tuned models and CDM paired with the base proposal $p^{\text{base}}(\mathbf{x}_{t-1}|\mathbf{x}_t)$ on the text and protein generation tasks. As diversity metrics, we report Self-BLEU [97] to evaluate text diversity by measuring the n-gram overlap ($n = 4$). Additionally, we report generative perplexity measured with GPT2-XL [62] to assess the quality of the generated texts. For protein generation, we report the number of clusters identified by FoldSeek [79] (Clusters) and the mean pairwise TMScore [93] (inner-TM). To ensure a fair comparison, we fix the inference compute budget across all methods. Fig. 3b presents the

diversity results for fine-tuned models compared to CDM. In the text generation task, both d1 [94] and DRAKES [82] exhibit mode collapse, as indicated by high Self-BLEU, along with degraded text quality, as reflected by high PPL. Notably, CDM maintains diversity with comparable reward. Similarly for protein generation, d1 suffers from mode collapse, while CDM mitigates mode collapse with comparable reward.

6.3 Twist Training Comparison

In this section, we compare the training dynamics of Soft Value and CDM on toxic text and DNA sequence generation, plotting reward against wall-clock training time with fixed training parameters (e.g., optimizer, architecture, batch size). Additionally, for the Soft Value [43] baseline, we sweep the Monte Carlo sample size M used to estimate the optimal twist function during training. The training curves on toxic text generation are shown in Fig. 4 where CDM converges faster in wall-clock time than Soft Value with various M . We attribute this superior convergence to the use of contrastive learning with negative samples, which has been shown to be highly effective in practice [96, 16, 7]. We provide the training dynamics for the regulatory DNA design task in Appendix E, which exhibit a similar pattern.

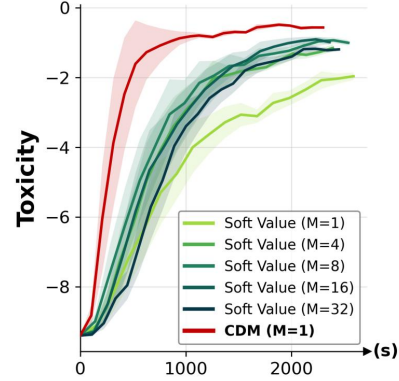


Figure 4: **Training Comparison of CDM with Soft Value.**

7 Conclusion

In this work, we presented Contrastive Distribution Matching (CDM), a framework that enables amortized Sequential Monte Carlo (SMC) inference in discrete diffusion by learning twist functions via a contrastive objective. We identified the reliance on costly Monte Carlo estimation as the primary bottleneck in twisted SMC inference and introduced a contrastive learning objective that leverages the diffusion forward process for efficient training. Our empirical results demonstrate that CDM achieves superior scaling performance and more efficient training compared to the baselines. Furthermore, we showed that CDM yields synergistic gains when integrated with fine-tuned proposals.

Acknowledgments

We thank Jason Yoo for insightful discussions on learning the value function for SMC in generative models, and Yuchen Zhu for providing new insights into the role of negative gradients.

References

- [1] Michael Samuel Albergo and Eric Vanden-Eijnden. Nets: A non-equilibrium transport sampler. In *International Conference on Machine Learning*, pages 1026–1055. PMLR, 2025.
- [2] Žiga Avsec, Vikram Agarwal, Daniel Visentin, Joseph R Ledsam, Agnieszka Grabska-Barwinska, Kyle R Taylor, Yannis Assael, John Jumper, Pushmeet Kohli, and David R Kelley. Effective gene expression prediction from sequence by integrating long-range interactions. *Nature methods*, 18(10):1196–1203, 2021.
- [3] Arpit Bansal, Hong-Min Chu, Avi Schwarzschild, Soumyadip Sengupta, Micah Goldblum, Jonas Geiping, and Tom Goldstein. Universal guidance for diffusion models. In *CVPRW*, 2023.
- [4] Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning. *arXiv*, 2024.
- [5] Andrew Campbell, Joe Benton, Valentin De Bortoli, Thomas Rainforth, George Deligiannidis, and Arnaud Doucet. A continuous time framework for discrete denoising models. *Advances in Neural Information Processing Systems*, 2022.
- [6] Gabriel Cardoso, Yazid Janati El Idrissi, Sylvain Le Corff, and Eric Moulines. Monte carlo guided diffusion for bayesian linear inverse problems. In *ICLR*, 2024.

- [7] Huayu Chen, Kaiwen Zheng, Qinsheng Zhang, Ganqu Cui, Yin Cui, Haotian Ye, Tsung-Yi Lin, Ming-Yu Liu, Jun Zhu, and Haoxiang Wang. Nft: Bridging supervised learning and reinforcement learning in math reasoning. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [8] Nicolas Chopin, Omiros Papaspiliopoulos, et al. *An introduction to sequential Monte Carlo*, volume 4. Springer, 2020.
- [9] Po-Wei Chou, Daniel Maturana, and Sebastian Scherer. Improving stochastic policy gradients in continuous control with deep reinforcement learning using the beta distribution. In *International conference on machine learning*, pages 834–843. PMLR, 2017.
- [10] Wenda Chu, Zihui Wu, Yifan Chen, Yang Song, and Yisong Yue. Split gibbs discrete diffusion posterior sampling. *arXiv preprint arXiv:2503.01161*, 2025.
- [11] Hyungjin Chung, Jeongsol Kim, Michael Thompson Mccann, Marc Louis Klasky, and Jong Chul Ye. Diffusion posterior sampling for general noisy inverse problems. In *ICLR*, 2023.
- [12] Kevin Clark, Paul Vicol, Kevin Swersky, and Fleet David J. Directly fine-tuning diffusion models on differentiable rewards. In *ICLR*, 2024.
- [13] Meihua Dang, Jiaqi Han, Minkai Xu, Kai Xu, Akash Srivastava, and Stefano Ermon. Inference-time scaling of diffusion language models with particle gibbs sampling. *arXiv preprint arXiv:2507.08390*, 2025.
- [14] Pierre Del Moral, Arnaud Doucet, and Ajay Jasra. Sequential monte carlo samplers. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 68(3):411–436, 2006.
- [15] Daryna Dementieva, Daniil Moskovskiy, Nikolay Babakov, Abinew Ali Ayele, Naqee Rizwan, Florian Schneider, Xintong Wang, Seid Muhie Yimam, Dmitry Ustalov, Elisei Stakovskii, et al. Overview of the multilingual text detoxification task at pan 2024. In *CLEF (Working Notes)*, pages 2432–2461, 2024.
- [16] Mingyang Deng, He Li, Tianhong Li, Yilun Du, and Kaiming He. Generative modeling via drifting. *arXiv preprint arXiv:2602.04770*, 2026.
- [17] Carles Domingo-Enrich, Michal Drozdal, Brian Karrer, and Ricky TQ Chen. Adjoint matching: Fine-tuning flow and diffusion generative models with memoryless stochastic optimal control. *arXiv preprint arXiv:2409.08861*, 2024.
- [18] Arnaud Doucet, Nando De Freitas, and Neil Gordon. An introduction to sequential monte carlo methods. In *Sequential Monte Carlo methods in practice*, pages 3–14. Springer, 2001.
- [19] Arnaud Doucet, Nando De Freitas, Neil James Gordon, et al. *Sequential Monte Carlo methods in practice*. Springer, 2001.
- [20] Bradley Efron. Tweedie’s formula and selection bias. *Journal of the American Statistical Association*, 106(496):1602–1614, 2011.
- [21] Ying Fan, Olivia Watkins, Yuqing Du, Hao Liu, Moonkyung Ryu, Craig Boutilier, Pieter Abbeel, Mohammad Ghavamzadeh, Kangwook Lee, and Kimin Lee. Dpok: reinforcement learning for fine-tuning text-to-image diffusion models. In *NeurIPS*, 2023.
- [22] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *ICML*, 2023.
- [23] Aaron Gokaslan and Vanya Cohen. Openwebtext corpus. <http://Skylion007.github.io/OpenWebTextCorpus>, 2019.
- [24] Sager J Gosai, Rodrigo I Castro, Natalia Fuentes, John C Butts, Susan Kales, Ramil R Noche, Kousuke Mouri, Pardis C Sabeti, Steven K Reilly, and Ryan Tewhey. Machine-guided design of synthetic cell type-specific cis-regulatory elements. *bioRxiv*, 2023.
- [25] Will Grathwohl, Kevin Swersky, Milad Hashemi, David Duvenaud, and Chris Maddison. Oops i took a gradient: Scalable sampling for discrete distributions. In *International Conference on Machine Learning*, pages 3831–3841. PMLR, 2021.
- [26] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

- [27] Mohsin Hasan, Viktor Ohanesian, Artem Gazizov, Yoshua Bengio, Alán Aspuru-Guzik, Roberto Bondesan, Marta Skreta, and Kirill Neklyudov. Discrete feynman-kac correctors. *arXiv preprint arXiv:2601.10403*, 2026.
- [28] Aaron Havens, Benjamin Kurt Miller, Bing Yan, Carles Domingo-Enrich, Anuroop Sriram, Brandon Wood, Daniel Levine, Bin Hu, Brandon Amos, Brian Karrer, et al. Adjoint sampling: Highly scalable diffusion samplers via adjoint matching. *arXiv preprint arXiv:2504.11713*, 2025.
- [29] Peter Holderrieth, Michael S Albergo, and Tommi Jaakkola. Leaps: A discrete neural sampler via locally equivariant networks. *arXiv preprint arXiv:2502.10843*, 2025.
- [30] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144*, 2016.
- [31] Natasha Jaques, Shixiang Gu, Dzmitry Bahdanau, José Miguel Hernández-Lobato, Richard E Turner, and Douglas Eck. Sequence tutor: Conservative fine-tuning of sequence generation models with kl-control. In *ICML*, 2017.
- [32] Hilbert J Kappen. Path integrals and symmetry breaking for optimal control theory. *Journal of statistical mechanics: theory and experiment*, 2005.
- [33] Jaihoon Kim, Taehoon Yoon, Jisung Hwang, and Minhyuk Sung. Inference-time scaling for flow models via stochastic generation and rollover budget forcing. *arXiv preprint arXiv:2503.19385*, 2025.
- [34] Sunwoo Kim, Minkyu Kim, and Dongmin Park. Test-time alignment of diffusion models without reward over-optimization. In *ICLR*, 2025.
- [35] Tomasz Korbak, Ethan Perez, and Christopher Buckley. RL with kl penalties is better viewed as bayesian inference. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 1083–1091, 2022.
- [36] Black Forest Labs. Flux. <https://github.com/black-forest-labs/flux>, 2024.
- [37] Black Forest Labs, Stephen Batifol, Andreas Blattmann, Frederic Boesel, Saksham Consul, Cyril Diagne, Tim Dockhorn, Jack English, Zion English, Patrick Esser, Sumith Kulal, Kyle Lacey, Yam Levi, Cheng Li, Dominik Lorenz, Jonas Müller, Dustin Podell, Robin Rombach, Harry Saini, Axel Sauer, and Luke Smith. Flux.1 kontext: Flow matching for in-context image generation and editing in latent space, 2025. URL <https://arxiv.org/abs/2506.15742>.
- [38] Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, et al. Rewardbench: Evaluating reward models for language modeling. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1755–1797, 2025.
- [39] Dieterich Lawson, Allan Raventós, Andrew Warrington, and Scott Linderman. Sixo: Smoothing inference with twisted objectives. In *NeurIPS*, 2022.
- [40] Chanhyuk Lee, Jaehoon Yoo, Manan Agarwal, Sheel Shah, Jerry Huang, Aditi Raghunathan, Seunghoon Hong, Nicholas M Boffi, and Jinwoo Kim. Flow map language models: One-step language modeling via continuous denoising. *arXiv preprint arXiv:2602.16813*, 2026.
- [41] Cheuk Kit Lee, Paul Jeha, Jes Frellsen, Pietro Lio, Michael Samuel Albergo, and Francisco Vargas. Debiasing guidance for discrete diffusion with sequential monte carlo. *arXiv preprint arXiv:2502.06079*, 2025.
- [42] Sergey Levine. Reinforcement learning and control as probabilistic inference: Tutorial and review. *arXiv*, 2018.
- [43] Xiner Li, Yulai Zhao, Chenyu Wang, Gabriele Scalia, Gokcen Eraslan, Surag Nair, Tommaso Biancalani, Aviv Regev, Sergey Levine, and Masatoshi Uehara. Derivative-free guidance in continuous and discrete diffusion models with soft value-based decoding. In *NeurIPS*, 2025.
- [44] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023.
- [45] Vasileios Lioutas, Jonathan Wilder Lavington, Justice Sefas, Matthew Niedoba, Yunpeng Liu, Berend Zwartsenberg, Setareh Dabiri, Frank Wood, and Adam Scibior. Critic sequential monte carlo. *arXiv preprint arXiv:2205.15460*, 2022.

- [46] Chris Yuhao Liu, Liang Zeng, Jiakai Liu, Rui Yan, Jujie He, Chaojie Wang, Shuicheng Yan, Yang Liu, and Yahui Zhou. Skywork-reward: Bag of tricks for reward modeling in llms. *arXiv preprint arXiv:2410.18451*, 2024.
- [47] Liyuan Liu, Chengyu Dong, Xiaodong Liu, Bin Yu, and Jianfeng Gao. Bridging discrete and backpropagation: Straight-through and beyond. In *NeurIPS*, 2023.
- [48] Varvara Logacheva, Daryna Dementieva, Sergey Ustyantsev, Daniil Moskovskiy, David Dale, Irina Krotova, Nikita Semenov, and Alexander Panchenko. Paradox: Detoxification with parallel data. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6804–6818, 2022.
- [49] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- [50] Nanye Ma, Shangyuan Tong, Haolin Jia, Hexiang Hu, Yu-Chuan Su, Mingda Zhang, Xuan Yang, Yandong Li, Tommi Jaakkola, Xuhui Jia, and Saining Xie. Inference-time scaling for diffusion models beyond scaling denoising steps. *arXiv*, 2025.
- [51] Sidharth Mudgal, Jong Lee, Harish Ganapathy, YaGuang Li, Tao Wang, Yanping Huang, Zhifeng Chen, Heng-Tze Cheng, Michael Collins, Trevor Strohman, et al. Controlled decoding from language models. *arXiv preprint arXiv:2310.17022*, 2023.
- [52] Christian A Naesseth, Fredrik Lindsten, Thomas B Schön, et al. Elements of sequential monte carlo. *Foundations and Trends in Machine Learning*, 12(3):307–392, 2019.
- [53] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, JUN ZHOU, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. In *NeurIPS*, 2025.
- [54] Hunter Nisonoff, Junhao Xiong, Stephan Allenspach, and Jennifer Listgarten. Unlocking guidance for discrete state-space diffusion and flow models. *arXiv preprint arXiv:2406.01572*, 2024.
- [55] Zijing Ou, Chinmay Pani, and Yingzhen Li. Inference-time scaling of discrete diffusion models via importance weighting and optimal proposal design. *arXiv preprint arXiv:2505.22524*, 2025.
- [56] Mingue Park, Jisung Hwang, Seungwoo Yoo, Kyeongmin Yeo, and Minhyuk Sung. Pairflow: Closed-form source-target coupling for few-step generation in discrete flow models. In *ICLR*, 2026.
- [57] Max Paulus, Dami Choi, Daniel Tarlow, Andreas Krause, and Chris J Maddison. Gradient estimation with stochastic softmax tricks. In *NeurIPS*, volume 33, pages 5691–5704, 2020.
- [58] Prin Phunaphibarn and Minhyuk Sung. Reward-guided discrete diffusion via clean-sample markov chain for molecule and biological sequence design. *arXiv preprint arXiv:2602.09424*, 2026.
- [59] Alexandre Piché, Valentin Thomas, Cyril Ibrahim, Yoshua Bengio, and Chris Pal. Probabilistic planning with sequential monte carlo methods. In *International Conference on Learning Representations*, 2018.
- [60] Peter Potapchik, Jason Yim, Adhi Saravanan, Peter Holderrieth, Eric Vanden-Eijnden, and Michael S Albergo. Discrete flow maps. *arXiv preprint arXiv:2604.09784*, 2026.
- [61] Mihir Prabhudesai, Russell Mendonca, Zheyang Qin, Katerina Fragkiadaki, and Deepak Pathak. Video diffusion alignment via reward gradients. *arXiv preprint arXiv:2407.08737*, 2024.
- [62] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [63] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *NIPS*, 2023.
- [64] Vignav Ramesh and Morteza Mardani. Test-time scaling of diffusion models via noise trajectory search. *arXiv preprint arXiv:2506.03164*, 2025.
- [65] Martin Raphan and Eero P Simoncelli. Least squares estimation without priors or supervision. *Neural computation*, 23(2):374–420, 2011.
- [66] Konrad Rawlik, Marc Toussaint, and Sethu Vijayakumar. On stochastic optimal control and reinforcement learning by approximate inference. *Proceedings of Robotics: Science and Systems VIII*, 2012.
- [67] Yinuo Ren, Wenhao Gao, Lexing Ying, Grant M Rotskoff, and Jiequn Han. Driftlite: Lightweight drift control for inference-time scaling of diffusion models. *arXiv preprint arXiv:2509.21655*, 2025.

- [68] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *CVPR*, 2022.
- [69] Subham S Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *NeurIPS*, 2024.
- [70] Anirban Sarkar, Ziqi Tang, Chris Z Zhao, and Peter K Koo. Designing DNA with tunable regulatory activity using discrete diffusion. In *NeurIPS 2024 Workshop on AI for New Drug Modalities*, 2024. URL <https://openreview.net/forum?id=Ioy8LCAyRj>.
- [71] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [72] Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis Titsias. Simplified and generalized masked diffusion for discrete data. In *NeurIPS*, 2024.
- [73] Raghav Singhal, Zachary Horvitz, Ryan Teehan, Mengye Ren, Zhou Yu, Kathleen McKeown, and Rajesh Ranganath. A general framework for inference-time scaling and steering of diffusion models. *arXiv preprint arXiv:2501.06848*, 2025.
- [74] Marta Skreta, Tara Akhound-Sadegh, Viktor Ohanesian, Roberto Bondesan, Alán Aspuru-Guzik, Arnaud Doucet, Rob Brekelmans, Alexander Tong, and Kirill Neklyudov. Feynman-kac correctors in diffusion: Annealing, guidance, and product of experts. *arXiv preprint arXiv:2503.02819*, 2025.
- [75] Oswin So, Brian Karrer, Chuchu Fan, Ricky TQ Chen, and Guan-Hong Liu. Discrete adjoint matching. *arXiv preprint arXiv:2602.07132*, 2026.
- [76] Jiaming Song, Arash Vahdat, Morteza Mardani, and Jan Kautz. Pseudoinverse-guided diffusion models for inverse problems. In *International conference on learning representations*, 2023.
- [77] Masatoshi Uehara, Yulai Zhao, Tommaso Biancalani, and Sergey Levine. Understanding reinforcement learning-based fine-tuning of diffusion models: A tutorial and review. *arXiv preprint arXiv:2407.13734*, 2024.
- [78] Masatoshi Uehara, Yulai Zhao, Kevin Black, Ehsan Hajiramezanali, Gabriele Scalia, Nathaniel Lee Diamant, Alex M Tseng, Tommaso Biancalani, and Sergey Levine. Fine-tuning of continuous-time diffusion models as entropy-regularized control. *arXiv*, 2024.
- [79] Michel Van Kempen, Stephanie S Kim, Charlotte Tumescheit, Milot Mirdita, Jeongjae Lee, Cameron LM Gilchrist, Johannes Söding, and Martin Steinegger. Fast and accurate protein structure search with foldseek. *Nature biotechnology*, 42(2):243–246, 2024.
- [80] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [81] Bram Wallace, Meihua Dang, Rafael Rafailov, Linqi Zhou, Aaron Lou, Senthil Purushwalkam, Stefano Ermon, Caiming Xiong, Shafiq Joty, and Nikhil Naik. Diffusion model alignment using direct preference optimization. In *CVPR*, 2024.
- [82] Chenyu Wang, Masatoshi Uehara, Yichun He, Amy Wang, Tommaso Biancalani, Avantika Lal, Tommi Jaakkola, Sergey Levine, Hanchen Wang, and Aviv Regev. Fine-tuning discrete diffusion models via reward optimization with applications to dna and protein design. *arXiv preprint arXiv:2410.13643*, 2024.
- [83] Haoxiang Wang, Wei Xiong, Tengyang Xie, Han Zhao, and Tong Zhang. Interpretable preferences via multi-objective reward modeling and mixture-of-experts. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 10582–10592, 2024.
- [84] Xinyou Wang, Zaixiang Zheng, Dongyu Xue, Shujian Huang, Quanquan Gu, et al. Dplm-2: A multimodal diffusion protein language model. In *ICLR*, 2025.
- [85] Luhuan Wu, Brian L Trippe, Christian Naesseth, David Blei, and John P Cunningham. Practical and asymptotically exact conditional sampling in diffusion models. In *NeurIPS*, 2023.
- [86] Kevin Yang and Dan Klein. FUDGE: Controlled text generation with future discriminators. In *NAACL*, 2021.

- [87] Ling Yang, Ye Tian, Bowen Li, Xincheng Zhang, Ke Shen, Yunhai Tong, and Mengdi Wang. MMaDA: Multimodal large diffusion language models. In *NeurIPS*, 2025.
- [88] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.
- [89] Taehoon Yoon, Yunhong Min, Kyeongmin Yeo, and Minhyuk Sung. Psi-sampler: Initial particle sampling for smc-based inference-time reward alignment in score models. *arXiv preprint arXiv:2506.01320*, 2025.
- [90] Jiwen Yu, Yinhuai Wang, Chen Zhao, Bernard Ghanem, and Jian Zhang. Freedom: Training-free energy-guided conditional diffusion model. In *ICCV*, 2023.
- [91] Oussama Zekri and Nicolas Boullé. Fine-tuning discrete diffusion models with policy gradient methods. *arXiv preprint arXiv:2502.01384*, 2025.
- [92] Ruqi Zhang, Xingchao Liu, and Qiang Liu. A langevin-like sampler for discrete distributions. In *International Conference on Machine Learning*, pages 26375–26396. PMLR, 2022.
- [93] Yang Zhang and Jeffrey Skolnick. Tm-align: a protein structure alignment algorithm based on the tm-score. *Nucleic acids research*, 33(7):2302–2309, 2005.
- [94] Siyan Zhao, Devaansh Gupta, Qinqing Zheng, and Aditya Grover. d1: Scaling reasoning in diffusion large language models via reinforcement learning. *arXiv preprint arXiv:2504.12216*, 2025.
- [95] Stephen Zhao, Rob Brekelmans, Alireza Makhzani, and Roger Grosse. Probabilistic inference in language models via twisted sequential monte carlo. *arXiv preprint arXiv:2404.17546*, 2024.
- [96] Kaiwen Zheng, Huayu Chen, Haotian Ye, Haoxiang Wang, Qinsheng Zhang, Kai Jiang, Hang Su, Stefano Ermon, Jun Zhu, and Ming-Yu Liu. Diffusionnft: Online diffusion reinforcement with forward process. *arXiv preprint arXiv:2509.16117*, 2025.
- [97] Yaoming Zhu, Sidi Lu, Lei Zheng, Jiaxian Guo, Weinan Zhang, Jun Wang, and Yong Yu. Texus: A benchmarking platform for text generation models. In *The 41st international ACM SIGIR conference on research & development in information retrieval*, pages 1097–1100, 2018.
- [98] Yuchen Zhu, Wei Guo, Jaemoon Choi, Guan-Hong Liu, Yongxin Chen, and Molei Tao. Mdns: Masked diffusion neural sampler via stochastic optimal control. *arXiv preprint arXiv:2508.10684*, 2025.
- [99] Yuchen Zhu, Wei Guo, Jaemoon Choi, Petr Molodyk, Bo Yuan, Molei Tao, and Yongxin Chen. Enhancing reasoning for diffusion llms via distribution matching policy optimization. *arXiv preprint arXiv:2510.08233*, 2025.

Appendix

Societal Impacts

In this work, we adapt CDM to sample from a reward-tilted distribution to optimize downstream objectives. We evaluate our method on toxic text generation strictly for safety benchmarking. Notably, this same technique can be inverted via a negative reward to actively reduce toxicity and mitigate harmful model behaviors. Furthermore, our application to protein generation holds the potential to accelerate therapeutic development and drug discovery. Lastly, applying our method to LLM alignment enhances the reliability and helpfulness of model interactions for end users.

Limitations and Future Works

While our current work investigates different twist function architectures, modeling highly complex rewards may require more advanced structural choices. Furthermore, learning expected future rewards in settings with extremely sparse signals still poses a significant challenge. In future work, we aim to extend our framework to natively support binary and categorical reward structures.

A Derivations

We consider the problem of sampling from the intermediate target distribution $p_t^*(\mathbf{x}_t) = \frac{1}{Z_t} p_t^{\text{base}}(\mathbf{x}_t) \psi_t^*(\mathbf{x}_t)$, and denote its unnormalized density by $\tilde{p}_t^*(\mathbf{x}_t) = p_t^{\text{base}}(\mathbf{x}_t) \psi_t^*(\mathbf{x}_t)$. For use in importance sampling (IS) and Sequential Monte Carlo (SMC), we lift this marginal target to a trajectory-level target. Let $\tilde{\pi}_{t:T}(\mathbf{x}_{t:T})$ be an arbitrary unnormalized path-space extension whose marginal over future variables recovers the unnormalized target at time t :

$$\sum_{\mathbf{x}_{t+1:T}} \tilde{\pi}_{t:T}(\mathbf{x}_{t:T}) = \tilde{p}_t^*(\mathbf{x}_t).$$

Equivalently, after normalization, the corresponding trajectory-level distribution has time- t marginal $p_t^*(\mathbf{x}_t)$. This formulation allows us to apply IS and SMC by sampling trajectories $\mathbf{x}_{t:T}$ from a proposal distribution and using their time- t components to approximate the desired marginal target.

A.1 Importance Sampling and Sequential Monte Carlo

Importance Sampling. Given a proposal distribution $q_{t:T}(\mathbf{x}_{t:T})$, IS corrects for the distributional mismatch with the target distribution $p_{t:T}^*$ using importance weights. The unnormalized importance weights $\{W_t\}_{k=1}^K$ and the self-normalized importance weights $\{\tilde{W}_t\}_{k=1}^K$ are computed as:

$$W_t^{(k)} = \frac{\tilde{\pi}_{t:T}(\mathbf{x}_{t:T}^{(k)})}{q_{t:T}(\mathbf{x}_{t:T}^{(k)})}, \quad \tilde{W}_t^{(k)} = \frac{W_t^{(k)}}{\sum_{j=1}^K W_t^{(j)}} \quad (12)$$

The normalized weights $\{\tilde{W}_t\}_{k=1}^K$ define an empirical distribution that enables approximate sampling from the target. However, IS often suffers from weight degeneracy problem and results in high-variance estimators.

Sequential Monte Carlo. To mitigate weight degeneracy, SMC interleaves sequential importance sampling with particle resampling. When extending a trajectory from $\mathbf{x}_{t:T}$ to $\mathbf{x}_{t-1:T}$, the incremental importance weight w_{t-1} is computed as:

$$w_{t-1} = \frac{\tilde{\pi}_{t-1:T}(\mathbf{x}_{t-1:T})}{\tilde{\pi}_{t:T}(\mathbf{x}_{t:T}) q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)}. \quad (13)$$

The unnormalized importance weight is then updated by multiplying the previous weight by an incremental importance weight: $W_{t-1} = W_t \cdot w_{t-1}$.

Under a Markovian assumption on the target trajectory, the joint distributions factorize as

$$\tilde{\pi}_{t-1:T}(\mathbf{x}_{t-1:T}) = \tilde{p}_{t-1}(\mathbf{x}_{t-1}) \prod_{s=t}^T \gamma_s(\mathbf{x}_s | \mathbf{x}_{s-1}), \quad (14)$$

$$\tilde{\pi}_{t:T}(\mathbf{x}_{t:T}) = \tilde{p}_t(\mathbf{x}_t) \prod_{s=t+1}^T \gamma_s(\mathbf{x}_s | \mathbf{x}_{s-1}), \quad (15)$$

where $\gamma_s(\mathbf{x}_s | \mathbf{x}_{s-1})$ is an arbitrary forward kernel.

Substituting Eqs. (14) and (15) into Eq. (13), the product over $s = t + 1, \dots, T$ cancels between numerator and denominator, leaving only the $s = t$ factor from the numerator:

$$w_{t-1} = \frac{\tilde{p}_{t-1}(\mathbf{x}_{t-1}) \prod_{s=t}^T \gamma_s(\mathbf{x}_s | \mathbf{x}_{s-1})}{\tilde{p}_t(\mathbf{x}_t) \prod_{s=t+1}^T \gamma_s(\mathbf{x}_s | \mathbf{x}_{s-1}) q(\mathbf{x}_{t-1} | \mathbf{x}_t)} = \frac{\tilde{p}_{t-1}(\mathbf{x}_{t-1})}{\tilde{p}_t(\mathbf{x}_t)} \frac{\gamma_t(\mathbf{x}_t | \mathbf{x}_{t-1})}{q(\mathbf{x}_{t-1} | \mathbf{x}_t)}. \quad (16)$$

Specifically, we choose the forward kernel to be the diffusion model's forward kernel [34, 55]:

$$\gamma_t(\mathbf{x}_t | \mathbf{x}_{t-1}) := p^{\text{base}}(\mathbf{x}_t | \mathbf{x}_{t-1}) = \frac{p^{\text{base}}(\mathbf{x}_{t-1} | \mathbf{x}_t) p_t^{\text{base}}(\mathbf{x}_t)}{p_{t-1}^{\text{base}}(\mathbf{x}_{t-1})}. \quad (17)$$

Under this choice, the unnormalized trajectory-level target reduces to $p_{t:T}^{\text{base}}(\mathbf{x}_{t:T}) \psi_t^*(\mathbf{x}_t)$, which is the form used in Eq. (6).

Substituting Eq. (17) into Eq. (16) gives:

$$w_{t-1} = \frac{p_{t-1}^{\text{base}}(\mathbf{x}_{t-1}) \psi_{t-1}^*(\mathbf{x}_{t-1})}{p_t^{\text{base}}(\mathbf{x}_t) \psi_t^*(\mathbf{x}_t)} \frac{p^{\text{base}}(\mathbf{x}_{t-1} | \mathbf{x}_t) p_t^{\text{base}}(\mathbf{x}_t)}{p_{t-1}^{\text{base}}(\mathbf{x}_{t-1}) q(\mathbf{x}_{t-1} | \mathbf{x}_t)}, \quad (18)$$

where the base marginals $p_{t-1}^{\text{base}}(\mathbf{x}_{t-1})$ and $p_t^{\text{base}}(\mathbf{x}_t)$ cancel out, leading to,

$$w_{t-1} = \frac{\psi_{t-1}^*(\mathbf{x}_{t-1})}{\psi_t^*(\mathbf{x}_t)} \frac{p^{\text{base}}(\mathbf{x}_{t-1} | \mathbf{x}_t)}{q(\mathbf{x}_{t-1} | \mathbf{x}_t)}. \quad (19)$$

After computing incremental weights, SMC resamples particles by drawing ancestor indices from the categorical distribution defined by the normalized weights:

$$a_{t-1}^{(k)} \sim \text{Cat}\left(\left\{\tilde{W}_{t-1}^{(i)}\right\}_{i=1}^K\right), \quad \tilde{W}_{t-1}^{(i)} = \frac{W_{t-1}^{(i)}}{\sum_{j=1}^K W_{t-1}^{(j)}}, \quad (20)$$

and replaces each particle by its sampled ancestor, $\mathbf{x}_{t-1}^{(k)} \leftarrow \mathbf{x}_{t-1}^{(a_{t-1}^{(k)})}$. In practice, resampling is triggered adaptively based on the effective sample size $\text{ESS} = 1 / \sum_{k=1}^K (\tilde{W}_t^{(k)})^2$, with resampling performed whenever ESS falls below a threshold [8]. After resampling, the unnormalized weights are reset to $W_{t-1}^{(k)} = 1$ for all particles. Setting $\text{ESS}_{\text{thres}} = 0$ disables resampling at every step, reducing SMC to IS. We present the full inference procedure in Alg. 1.

The resulting particle system yields asymptotically consistent estimators of expectations under the intermediate target p_t^* , together with an empirical approximation of p_t^* itself:

$$\mathbb{E}_{p_t^*}[f(\mathbf{x}_t)] \approx \sum_{k=1}^K \tilde{W}_t^{(k)} f(\mathbf{x}_t^{(k)}), \quad p_t^* \approx \sum_{k=1}^K \tilde{W}_t^{(k)} \delta_{\mathbf{x}_t^{(k)}}. \quad (21)$$

A.2 Contrastive Gradient Derivations

We provide the derivation of the contrastive gradient Eq. (10) for completeness. A formulation within the autoregressive language model is demonstrated in [95].

Plugging the parameterized tilted distribution from Eq. (8) into the forward KL objective yields:

$$\mathcal{D}_{\text{KL}}(p_t^*(\mathbf{x}_t) \parallel p_t^\phi(\mathbf{x}_t)) = \mathbb{E}_{p_t^*(\mathbf{x}_t)} \left[\log p_t^*(\mathbf{x}_t) - \log p_t^\phi(\mathbf{x}_t) \right] \quad (22)$$

$$= \mathbb{E}_{p_t^*(\mathbf{x}_t)} \left[\log p_t^*(\mathbf{x}_t) - \log p_t^{\text{base}}(\mathbf{x}_t) - \log \psi_t^\phi(\mathbf{x}_t) \right] + \log \mathcal{Z}_t^\phi. \quad (23)$$

Noting that the first two terms in the expectation are invariant to ϕ , the gradient with respect to the parameters ϕ simplifies to:

$$\nabla_{\phi} \mathcal{D}_{\text{KL}}(p_t^* \| p_t^{\phi}) = -\mathbb{E}_{p_t^*(\mathbf{x}_t)} \left[\nabla_{\phi} \log \psi_t^{\phi}(\mathbf{x}_t) \right] + \nabla_{\phi} \log \mathcal{Z}_t^{\phi}. \quad (24)$$

With log-derivative trick, the normalization constant expands as follows:

$$\nabla_{\phi} \log \mathcal{Z}_t^{\phi} = \frac{1}{\mathcal{Z}_t^{\phi}} \nabla_{\phi} \sum_{\mathbf{x}_t} p_t^{\text{base}}(\mathbf{x}_t) \psi_t^{\phi}(\mathbf{x}_t) \quad (25)$$

$$= \sum_{\mathbf{x}_t} \frac{p_t^{\text{base}}(\mathbf{x}_t) \psi_t^{\phi}(\mathbf{x}_t)}{\mathcal{Z}_t^{\phi}} \nabla_{\phi} \log \psi_t^{\phi}(\mathbf{x}_t) = \mathbb{E}_{p_t^{\phi}(\mathbf{x}_t)} \left[\nabla_{\phi} \log \psi_t^{\phi}(\mathbf{x}_t) \right], \quad (26)$$

where the last equality uses the definition of p_t^{ϕ} in Eq. (8).

Substituting this result into Eq. (24) and taking the expectation over time t yields the contrastive gradient in Eq. (10):

$$-\nabla_{\phi} \mathcal{L}_{\text{CDM}}(\phi) = \mathbb{E}_t \left[\mathbb{E}_{p_t^*(\mathbf{x}_t)} \left[\nabla_{\phi} \log \psi_t^{\phi}(\mathbf{x}_t) \right] - \mathbb{E}_{p_t^{\phi}(\mathbf{x}_t)} \left[\nabla_{\phi} \log \psi_t^{\phi}(\mathbf{x}_t) \right] \right]. \quad (27)$$

A.3 Forward-Based Gradient Estimation.

In this section, we show that the positive-term gradient estimator used in Eq. (11),

$$\mathbb{E}_{p_0^*(\mathbf{x}_0)} \mathbb{E}_{p^{\text{base}}(\mathbf{x}_t | \mathbf{x}_0)} \left[\nabla_{\phi} \log \psi_t^{\phi}(\mathbf{x}_t) \right], \quad (28)$$

is identical to the marginal expectation

$$\mathbb{E}_{p_t^*(\mathbf{x}_t)} \left[\nabla_{\phi} \log \psi_t^{\phi}(\mathbf{x}_t) \right]. \quad (29)$$

The KL-regularized objective in Eq. (3) admits the following closed-form solution over joint trajectory distributions [78, 77]:

$$p_{0:T}^*(\mathbf{x}_{0:T}) = \frac{1}{\mathcal{Z}} p_{0:T}^{\text{base}}(\mathbf{x}_{0:T}) \exp \left(\frac{r(\mathbf{x}_0)}{\beta} \right), \quad \mathcal{Z} = \mathbb{E}_{p_0^{\text{base}}(\mathbf{x}_0)} [\exp(r(\mathbf{x}_0)/\beta)]. \quad (30)$$

This closed-form solution follows from a standard variational argument. For any trajectory distribution $q_{0:T}$, consider the functional $\mathcal{J}$ given as:

$$\mathcal{J}(q_{0:T}) = \sum_{\mathbf{x}_{0:T}} q_{0:T}(\mathbf{x}_{0:T}) r(\mathbf{x}_0) - \beta \sum_{\mathbf{x}_{0:T}} q_{0:T}(\mathbf{x}_{0:T}) \log \frac{q_{0:T}(\mathbf{x}_{0:T})}{p_{0:T}^{\text{base}}(\mathbf{x}_{0:T})}. \quad (31)$$

Introducing a Lagrange multiplier λ for the constraint $\sum_{\mathbf{x}_{0:T}} q_{0:T}(\mathbf{x}_{0:T}) = 1$, the stationarity condition gives

$$r(\mathbf{x}_0) - \beta \left(\log \frac{q_{0:T}(\mathbf{x}_{0:T})}{p_{0:T}^{\text{base}}(\mathbf{x}_{0:T})} + 1 \right) + \lambda = 0. \quad (32)$$

Rearranging yields $q_{0:T}(\mathbf{x}_{0:T}) \propto p_{0:T}^{\text{base}}(\mathbf{x}_{0:T}) \exp \left(\frac{r(\mathbf{x}_0)}{\beta} \right)$. Normalizing this density gives the unique optimizer of the KL-regularized objective:

$$p_{0:T}^*(\mathbf{x}_{0:T}) = \frac{1}{\mathcal{Z}} p_{0:T}^{\text{base}}(\mathbf{x}_{0:T}) \exp \left(\frac{r(\mathbf{x}_0)}{\beta} \right), \quad (33)$$

where

$$\mathcal{Z} = \sum_{\mathbf{x}_{0:T}} p_{0:T}^{\text{base}}(\mathbf{x}_{0:T}) \exp \left(\frac{r(\mathbf{x}_0)}{\beta} \right) = \mathbb{E}_{p_0^{\text{base}}(\mathbf{x}_0)} \left[\exp \left(\frac{r(\mathbf{x}_0)}{\beta} \right) \right]. \quad (34)$$

Decomposing $p_{0:T}^{\text{base}}(\mathbf{x}_{0:T}) = p_0^{\text{base}}(\mathbf{x}_0) p^{\text{base}}(\mathbf{x}_{1:T} | \mathbf{x}_0)$ and collecting the reward-dependent terms into the clean marginal in Eq. (33), we obtain:

$$p_{0:T}^*(\mathbf{x}_{0:T}) = \underbrace{p_0^{\text{base}}(\mathbf{x}_0) \exp(r(\mathbf{x}_0)/\beta)}_{= p_0^*(\mathbf{x}_0)} p^{\text{base}}(\mathbf{x}_{1:T} | \mathbf{x}_0). \quad (35)$$

Since $p_{0:T}^*(\mathbf{x}_{0:T})$ also factorizes as $p_0^*(\mathbf{x}_0) p^*(\mathbf{x}_{1:T} | \mathbf{x}_0)$, comparing the two expressions gives

$$p^*(\mathbf{x}_{1:T} | \mathbf{x}_0) = p^{\text{base}}(\mathbf{x}_{1:T} | \mathbf{x}_0). \quad (36)$$

Marginalizing Eq. (36) over $(\mathbf{x}_{1:t-1}, \mathbf{x}_{t+1:T})$ for any $t \geq 1$ gives the per-step forward marginal:

$$p^*(\mathbf{x}_t | \mathbf{x}_0) = \sum_{\substack{\mathbf{x}_{1:t-1}, \\ \mathbf{x}_{t+1:T}}} p^*(\mathbf{x}_{1:T} | \mathbf{x}_0) = \sum_{\substack{\mathbf{x}_{1:t-1}, \\ \mathbf{x}_{t+1:T}}} p^{\text{base}}(\mathbf{x}_{1:T} | \mathbf{x}_0) = p^{\text{base}}(\mathbf{x}_t | \mathbf{x}_0). \quad (37)$$

Using Eq. (37), the target joint marginal over $(\mathbf{x}_0, \mathbf{x}_t)$ factorizes as $p_{0,t}^*(\mathbf{x}_0, \mathbf{x}_t) = p_0^*(\mathbf{x}_0) p^{\text{base}}(\mathbf{x}_t | \mathbf{x}_0)$. Therefore, marginalizing over $\mathbf{x}_0$ yields

$$p_t^*(\mathbf{x}_t) = \sum_{\mathbf{x}_0} p_0^*(\mathbf{x}_0) p^{\text{base}}(\mathbf{x}_t | \mathbf{x}_0). \quad (38)$$

Substituting Eq. (38) into the positive term of Eq. (10) yields:

$$\mathbb{E}_{p_t^*(\mathbf{x}_t)} \left[\nabla_{\phi} \log \psi_t^{\phi}(\mathbf{x}_t) \right] = \sum_{\mathbf{x}_t} p_t^*(\mathbf{x}_t) \nabla_{\phi} \log \psi_t^{\phi}(\mathbf{x}_t) \quad (39)$$

$$= \sum_{\mathbf{x}_t} \sum_{\mathbf{x}_0} p_0^*(\mathbf{x}_0) p^{\text{base}}(\mathbf{x}_t | \mathbf{x}_0) \nabla_{\phi} \log \psi_t^{\phi}(\mathbf{x}_t) \quad (40)$$

$$= \mathbb{E}_{p_0^*(\mathbf{x}_0)} \mathbb{E}_{p^{\text{base}}(\mathbf{x}_t | \mathbf{x}_0)} \left[\nabla_{\phi} \log \psi_t^{\phi}(\mathbf{x}_t) \right]. \quad (41)$$

This establishes the forward-based reformulation of the positive term used in Eq. (11).

B Algorithms

In this section, we present the algorithmic implementation of Twisted Sequential Monte Carlo (SMC) and the complete CDM training loop. The detailed pseudocode for both procedures is provided in Algs. 1 and 2.

B.1 Twisted SMC

As discussed in Sec. A.1, the algorithm interleaves sequential importance sampling with particle resampling based on the importance weights. To establish a unified algorithmic framework, Alg. 1 operates on a generic twist function, denoted as Ψ .

For positive sampling, Ψ is instantiated as the Monte Carlo estimate $\hat{\psi}$ defined in Eq. (7) to approximate the target distribution p^* . For the negative sampling, Ψ is instantiated as the parameterized twist network ψ^{ϕ} to sample from p^{ϕ} . Furthermore, the resampling behavior is governed by an Effective Sample Size (ESS) threshold, $\text{ESS}_{\text{thres}}$. Notably, by setting $\text{ESS}_{\text{thres}} = 0$, we explicitly disable the resampling mechanism, reducing SMC to standard Importance Sampling (IS). Note that for IS, the twist ratios $\Psi_{t-1}(\mathbf{x}_{t-1}^{(k)})/\Psi_t(\mathbf{x}_t^{(k)})$ telescope across timesteps and therefore do not need to be computed explicitly except at the final step t_{stop} . In particular, for $t < t_{\text{stop}}$, we may set $w_{t-1}^{(k)} = p^{\text{base}}(\mathbf{x}_{t-1}^{(k)} | \mathbf{x}_t^{(k)})/q(\mathbf{x}_{t-1}^{(k)} | \mathbf{x}_t^{(k)})$ and calculate the twist function only at the final step via $w_{t_{\text{stop}}}^{(k)} = \Psi_{t_{\text{stop}}}(\mathbf{x}_{t_{\text{stop}}}^{(k)}) p^{\text{base}}(\mathbf{x}_{t_{\text{stop}}}^{(k)} | \mathbf{x}_{t_{\text{stop}}+1}^{(k)})/q(\mathbf{x}_{t_{\text{stop}}}^{(k)} | \mathbf{x}_{t_{\text{stop}}+1}^{(k)})$, where we omitted $\Psi_T(\mathbf{x}_T^{(k)})$ since all $\{\mathbf{x}_T^{(k)}\}_{k=1}^K$ are at the same mask state $\mathbf{m}$. Fig. 5 compares standard SMC with the proposed amortized inference approach using a learned twist.

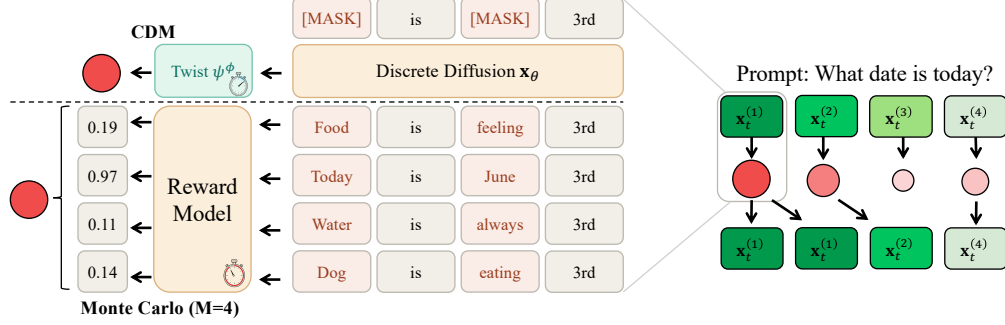


Figure 5: **Amortized Twisted SMC Procedure.** With the learned twist function, we can amortize the SMC inference with a single forward pass. On the other hand, SMC relies on expensive Monte Carlo estimate to approximate the twist function.

Algorithm 1: Twisted Sequential Monte Carlo / Importance Sampling

```

1 Function TwistSMC( $K, q, \Psi, \text{ESS}_{\text{thres}}, t_{\text{stop}}$ )
  // Inputs:
  //  $K$ : Number of particles    $q$ : Proposal distribution
  //  $\Psi$ : Twist function (MC estimate  $\hat{\psi}$  or network  $\psi^\phi$ )
  //  $\text{ESS}_{\text{thres}}$ : Resampling threshold (0 for IS without resampling)
  //  $t_{\text{stop}}$ : Stop timestep
2   $\{\mathbf{x}_T^{(k)}\}_{k=1}^K \sim p_T^{\text{base}}, \quad \{W_T^{(k)}\}_{k=1}^K \leftarrow 1$ 
3  for  $t = T, \dots, t_{\text{stop}} + 1$  do
4     $\{\mathbf{x}_{t-1}^{(k)}\}_{k=1}^K \sim \{q(\cdot | \mathbf{x}_t^{(k)})\}_{k=1}^K$ 
5     $\{w_{t-1}^{(k)}\}_{k=1}^K \leftarrow \left\{ \frac{\Psi_{t-1}(\mathbf{x}_{t-1}^{(k)}) p^{\text{base}}(\mathbf{x}_{t-1}^{(k)} | \mathbf{x}_t^{(k)})}{\Psi_t(\mathbf{x}_t^{(k)}) q(\mathbf{x}_{t-1}^{(k)} | \mathbf{x}_t^{(k)})} \right\}_{k=1}^K$  // Eq. (6)
6     $\{W_{t-1}^{(k)}\}_{k=1}^K \leftarrow \{W_t^{(k)} \cdot w_{t-1}^{(k)}\}_{k=1}^K$ 
7     $\{\tilde{W}_{t-1}^{(k)}\}_{k=1}^K \leftarrow \{W_{t-1}^{(k)} / \sum_{j=1}^K W_{t-1}^{(j)}\}_{k=1}^K$ 
8     $\text{ESS} \leftarrow 1 / \sum_{k=1}^K (\tilde{W}_{t-1}^{(k)})^2$ 
9    if  $\text{ESS} < \text{ESS}_{\text{thres}}$  then
10      $\{a_{t-1}^{(k)}\}_{k=1}^K \sim \text{Cat}(\{\tilde{W}_{t-1}^{(i)}\}_{i=1}^K)$ 
11      $\{\mathbf{x}_{t-1}^{(k)}\}_{k=1}^K \leftarrow \{\mathbf{x}_{t-1}^{(a_{t-1}^{(k)})}\}_{k=1}^K, \quad \{W_{t-1}^{(k)}\}_{k=1}^K \leftarrow 1$  // Resampling
12   end
13 end
14 return  $\{(\mathbf{x}_{t_{\text{stop}}}^{(k)}, \tilde{W}_{t_{\text{stop}}}^{(k)})\}_{k=1}^K$ 

```

B.2 CDM: Twist Training

Building on this SMC framework, Alg. 2 outlines the full training procedure for Contrastive Distribution Matching (CDM), which estimates the contrastive gradient in Eq. (11) efficiently by exploiting the diffusion forward process.

For positive sampling, we draw clean samples via SMC and store them in a buffer $\mathcal{B}$ of capacity B_{buffer} , refreshed every n_{update} steps. We then draw $\mathbf{x}_0^* \sim \mathcal{B}$ and apply the cheap forward kernel Eq. (1) to obtain $\mathbf{x}_t^*$, reusing each clean sample across many gradient updates. For the negative term, we

Algorithm 2: CDM: Contrastive Twist Training via Diffusion Forward Kernel

Input: Base model p^{base} , MC twist $\hat{\psi}$ Eq. (7), twist network ψ^ϕ , ESS threshold $\text{ESS}_{\text{thres}}$, positive buffer size B_{buffer} , update interval n_{update} , learning rate γ , EMA rate η , batch size B , $\bar{\phi} = \text{stopgrad}(\phi)$

Output: Optimized twist network parameters ϕ

```

1  $\phi_{\text{EMA}} \leftarrow \phi$ ,  $\text{step} \leftarrow 0$ ,  $\mathcal{B} \leftarrow \emptyset$ 
2 while not converged do
3    $\text{step} \leftarrow \text{step} + 1$ 
4    $t \sim \mathcal{U}(\{1, \dots, T\})$ 
5   if  $\text{step} \% n_{\text{update}} = 0$  or  $\mathcal{B} = \emptyset$  then
6      $\{(\mathbf{x}_0^{*(k)}, \tilde{W}_0^{*(k)})\}_{k=1}^{B_{\text{buffer}}} \leftarrow \text{TwistSMC}(B_{\text{buffer}}, p^{\text{base}}, \hat{\psi}, \text{ESS}_{\text{thres}}, 0)$  // Pos. Alg. 1
7      $\mathcal{B} \leftarrow \{(\mathbf{x}_0^{*(k)}, \tilde{W}_0^{*(k)})\}_{k=1}^{B_{\text{buffer}}}$ 
8   end
9    $\{(\mathbf{x}_0^{*(i)}, \tilde{W}_0^{*(i)})\}_{i=1}^B \sim \mathcal{B}$ 
10   $\{\mathbf{x}_t^{*(i)}\}_{i=1}^B \sim \{p^{\text{base}}(\cdot | \mathbf{x}_0^{*(i)})\}_{i=1}^B$  // Forward Eq. (1)
11   $\{(\mathbf{x}_t^{\bar{\phi}(i)}, \tilde{W}_t^{\phi(i)})\}_{i=1}^B \leftarrow \text{TwistSMC}(B, p^{\text{base}}, \psi^{\phi_{\text{EMA}}}, 0, t)$  // Neg. Alg. 1
12   $\nabla_{\phi} \mathcal{L}_{\text{CDM}} \leftarrow -\frac{1}{B} \sum_{i=1}^B \left( \tilde{W}_0^{*(i)} \nabla_{\phi} \log \psi_t^{\phi}(\mathbf{x}_t^{*(i)}) - \tilde{W}_t^{\phi(i)} \nabla_{\phi} \log \psi_t^{\phi}(\mathbf{x}_t^{\bar{\phi}(i)}) \right)$  // Eq. (11)
13   $\phi \leftarrow \phi - \gamma \nabla_{\phi} \mathcal{L}_{\text{CDM}}$ 
14   $\phi_{\text{EMA}} \leftarrow \eta \phi_{\text{EMA}} + (1 - \eta) \phi$ 
15 end
16 return  $\phi$ 

```

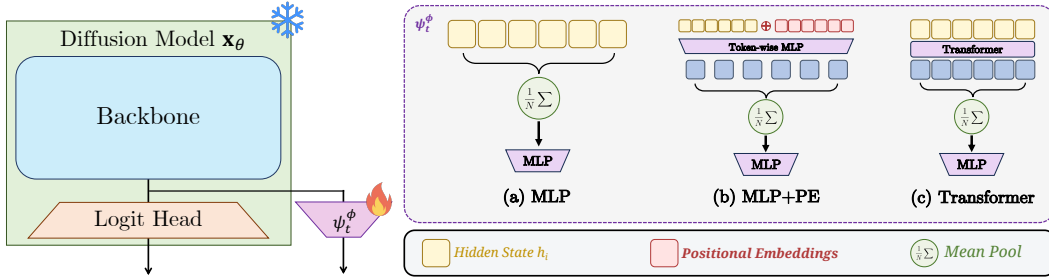


Figure 6: **Twist Head Architecture.** (Left) We parameterize the twisting function as a lightweight head that predicts the value based on the last hidden state of the denoising network. (Right) We consider three architectural choices for the twist head: (a) MLP, (b) MLP+PE, and (c) Transformer

draw $\mathbf{x}_t^\phi$ via importance sampling under the EMA-updated twist $\psi^{\phi_{\text{EMA}}}$. We present implementation details and ablations of the key hyperparameters in the next section.

C Implementation Details

In this section, we introduce choice of twist network parameterization and present the implementation details of the baselines including CDM.

	Base Model x_θ (ms)	Twist Head ψ^ϕ (ms)	Reward Model r (ms)	Base / Twist Ratio (%)
Toxic Text Generation (Sec. 6.1.1)	14.227	0.101	4.414	0.710
Regulatory DNA Design (Sec. 6.1.2)	9.829	0.289	11.438	2.940
Protein Designability (Sec. 6.1.3)	33.901	0.144	1356.0	0.425
dLLM Alignment (Sec. 6.1.4)	28.619	1.327	24.268	4.637

Table 1: **Twist Function Runtime Analysis.** We parameterize the twist function as a lightweight scalar head, incurring negligible runtime overhead.

	Toxic Text Generation	Regulatory DNA design	Protein Designability	dLLM Alignment
Sequence Length (N)	100	200	204	128
Diffusion Steps (T)	100	50	20	128
Norm. ESS ($\text{ESS}_{\text{thres}}$)	0.5	0.5	0.5	0.5
KL Weight (β)	0.2	0.2	0.1	2.0
Architecture	MLP	MLP + PE	MLP	Transformer
Optimizer	AdamW	AdamW	AdamW	AdamW
Learning Rate (γ)	1e−4	3e−4	1e−4	1e−4
Weight Decay	0.01	0.01	0.01	0.01
Buffer Size (B_{buffer})	64	64	64	64
Buffer Update Interval (n_{update})	4	4	10	4

Table 2: **Training and Inference Hyperparameters.** We fix the hyperparameters across all baselines to ensure a fair comparison. Soft Value and CDM use the same twist function parameterization, while CDM introduces additional hyperparameters for the positive buffer.

C.1 Twist Head Parameterization

As discussed in Sec. 4.3, instead of training a separate network from scratch for the twist function, we parameterize the twist function as a lightweight head as shown in Fig. 6. The twist head takes as input the last hidden state of the denoising network and outputs a scalar representing the value.

To validate our claim, we report a runtime analysis in Tab. 1, where we use NVIDIA PRO 6000 to measure the runtime. The twist head forward pass accounts for less than 5% of the base model runtime, and as little as 0.5% in some configurations. This efficiency follows from the parameterization itself, since a single forward pass through the denoising backbone produces a shared feature representation from which the logit head and twist head are computed in parallel. Consequently, evaluating ψ_t^ϕ during SMC adds negligible cost relative to standard sampling.

As shown in Fig. 6, we consider three different architectural choices for the twist head: MLP, MLP+PE, and Transformer.

MLP. The MLP takes as input the mean-pooled representation of the last hidden state of the denoising network.

MLP+PE. We incorporate positional embeddings into the twist model. Specifically, we take the final hidden representations from the denoising network and add sinusoidal positional embeddings to each token. A shared MLP head is then applied token-wise to produce positionally encoded token features. The resulting representations are mean pooled across tokens and passed through a final MLP.

Transformer. For tasks with stronger positional dependencies, we also consider a transformer-based architecture [80] to more effectively capture the positional dependencies.

For fair comparison, we use the same twist head architecture and shared training hyperparameters across Soft Value and CDM in the evaluations reported in Sec. 6.

Toxicity	$M = 1$	$M = 4$	Protein	$M = 1$	$M = 4$	Toxicity	Protein	
Soft Value	-1.8060	-0.9030	Soft Value	-1.8237	-1.6895	IS	-1.0367	-1.8049
CDM	-0.4614	-0.5079	CDM	-1.6950	-1.5725	SMC	-0.4614	-1.5725

(a) Monte Carlo Sample Size M (Train-Time)

(b) Positive Sampling Method

Table 3: **Choice of Monte-Carlo Sample Size M and Positive Sampling Method.** (a) We show that CDM outperforms Soft Value across various M . (b) While both IS and SMC can be used for sampling positive samples, we find that SMC yields better performance.

C.2 Experiment Details

Tab. 2 summarizes the hyperparameters used in each of the four experiments in Sec. 6. For CDM, we include the size of the positive buffer as well as the the positive buffer update interval n_{update} which specifies the number of training steps taken before the positive buffer is cleared and resampled.

For all experiments except the dLLM alignment task, we trained the twist estimator on a single NVIDIA RTX Pro 6000 GPU. For dLLM alignment, we used 4 NVIDIA B200 GPUs.

Toxic Text Generation. For the toxic text generation task, we use a publicly available pretrained MDM [69] trained on the OpenWebText dataset [23]. Following prior work [55], we set the KL weight to $\beta = 0.2$ and generate sequences of length 100 over 100 denoising steps. In this task, we use an MLP twist head with 5 layers and width 768.

Regulatory DNA Sequence Design. We train a CNN-parameterized MDM [69] on a publicly available enhancer activity dataset [24]. For all experiments, we generate DNA sequences with a fixed sequence length of 200, and run the diffusion process for 50 steps during sampling. Since the denoising backbone is CNN-parameterized, the twist head incorporates positional information explicitly via an MLP with sinusoidal positional encodings (MLP+PE), comprising 7 layers of width 1024.

Protein Designability. We evaluate our method on protein generation using DPLM-2 [84], a discrete diffusion model that jointly generates amino acid sequences and structural tokens. For all experiments, we generate proteins with a fixed sequence length of 100 residues (204 tokens total, including 100 sequence tokens, 100 structure tokens, and 4 `<bos>` and `<eos>` tokens for the sequence and structure tokens). The denoising step is set to 20, which we found to provide a good balance between generation quality and computational efficiency. We use a larger buffer update interval $n_{\text{update}} = 10$ in protein generation, since reward evaluation is expensive to amortize the cost of positive sampling over more gradient steps. We parameterize the twist head as a 5-layer MLP with width 1024. The twist model is trained using a learning rate of 1×10^{-4} and weight decay 0.01 using the AdamW optimizer [49].

dLLM Alignment. For the dLLM alignment task, we adopt the pretrained LLaDA-8B-Instruct [53] as our base diffusion language model. We train the twist estimator using prompts from Reward-Bench [38], randomly partitioned into an 80% training split and a 20% validation split for evaluation. In this experiment, we generate sequences of length 128 using 128 denoising steps and set the KL regularization weight to $\beta = 2.0$, a larger value than in the other applications because the reward model produces reward scores on a larger scale. We parameterize the twist head as a 2-layer Transformer [80] with hidden dimension 4096.

D Ablation Studies

In this section, we provide ablation and hyperparameter studies for CDM training.

Positive Buffer. As discussed in Sec. 6, for more efficient training, we reuse the positive samples by maintaining a buffer $\mathcal{B}$ and re-noising them across timesteps. The positive buffer is updated every n_{update} steps. Increasing n_{update} improves training efficiency by reusing positive samples obtained

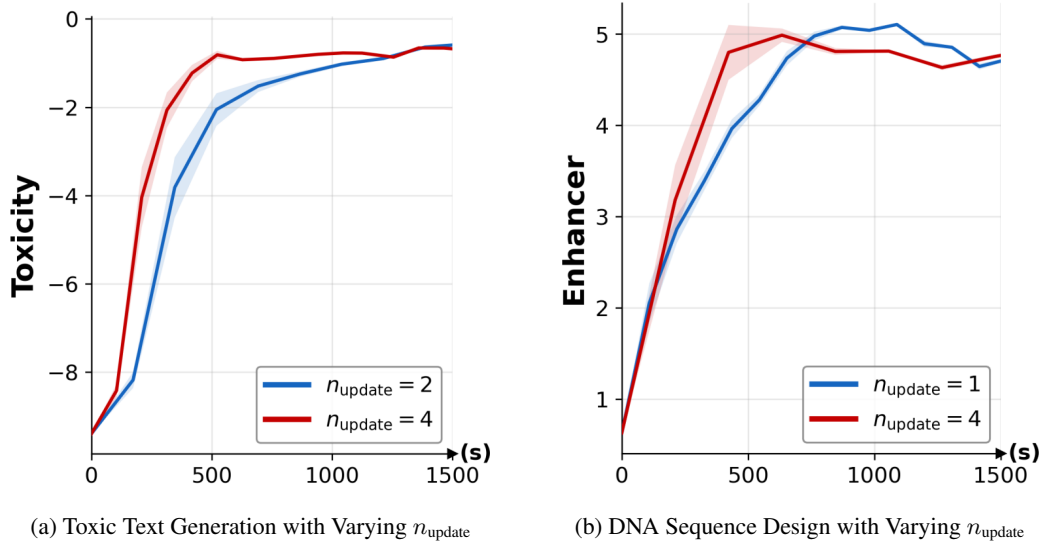


Figure 7: **Positive Buffer Ablation Results.** We present an ablation study on the buffer update frequency, n_{update} , evaluating its impact on both (a) toxic text generation and (b) regulatory DNA sequence design.

from SMC. In particular, when the reward is expensive, increasing n_{update} reduces the number of reward evaluations required. In Figs. 7a and 7b, we show that CDM performs well across various update intervals n_{update} .

Monte Carlo Sample Size. Although twist-learning methods like Soft Value and CDM bypass the need for Monte Carlo estimation at inference by amortizing the cost into a single forward pass, the number of Monte Carlo samples M remains as a hyperparameter during training. Tab. 3a presents the quantitative performance of both methods across varying values of M used for target estimation at train time. For Soft Value, increasing M initially yields performance gains. However, as shown in Fig. 4, these improvements quickly plateau for $M > 1$. In contrast, CDM maintains consistent performance across different values of M .

Choice of Sampling. As discussed in Sec. 4.1, positive sampling during the training of CDM can be performed using either standard Importance Sampling (IS) or Sequential Monte Carlo (SMC). As shown in Tab. 3b, using SMC for positive sampling consistently yields better downstream performance than IS under identical hyperparameter settings. We attribute this improvement to the resampling mechanism of SMC, which mitigates the severe weight degeneracy often observed in IS.

E Additional Results

In this section, we provide additional results on fine-tuned proposals and training convergence.

Additional Results on Compatibility with Fine-Tuned Proposal. For tasks with differentiable rewards, such as toxic text generation (Sec. 6.1.1), reward alignment can be achieved by fine-tuning the proposal via direct gradient backpropagation. To explore this regime, we additionally pair CDM with DRAKES [82], a method that fine-tunes a base model through direct backpropagation using the Gumbel-Softmax trick [30]. As shown in Fig. 8, applying CDM on top of DRAKES further improves its performance, outperforming all other baselines. This demonstrates the compatibility and synergistic potential of CDM when integrated with fine-tuned models.

Additional Results on Training Convergence. In addition to the quantitative results discussed in Sec. 6.3, we present comparison of the training dynamics of Soft Value and CDM for the regulatory DNA sequence design task in Fig. 9. We observe a consistent trend: Soft Value plateaus early in training, whereas CDM converges more efficiently and achieves a higher final reward.

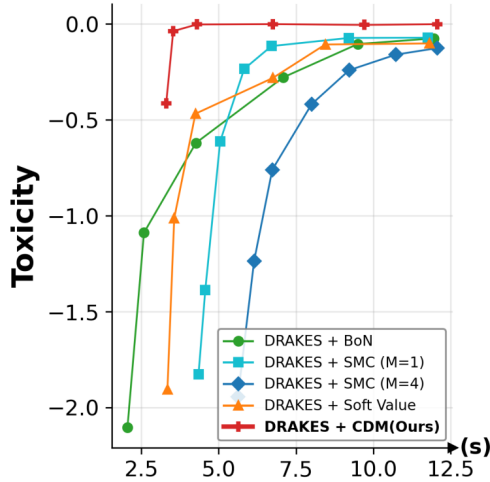


Figure 8: **Scaling with Direct Backpropagation Fine-Tuned Proposal [82].** CDM is also compatible with DRAKES [82], a proposal fine-tuned via direct backpropagation for tasks with differentiable rewards.

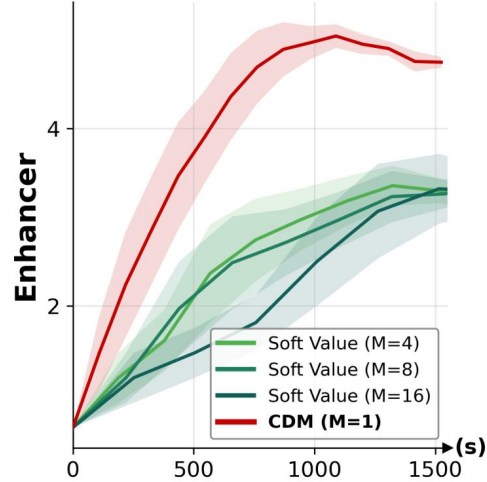


Figure 9: **Training Comparison of CDM and Soft Value for Regulatory DNA Sequence Design.** CDM shows faster training convergence and achieves higher reward than Soft Value with different M .

F Qualitative Results

In this section, we provide qualitative results for protein and text generation.

F.1 Protein Designability

Fig. 10 shows the qualitative results for protein design. The generated protein is shown in blue while the refolded protein obtained using ESMFold [44] is shown in orange. We aim to generate designable proteins for which the generated and refolded structures are well aligned. CDM achieves highly designable proteins, as shown by the overlapping and closely matching generated and refolded structures.

F.2 Diffusion Large Language Model Alignment

Tab. 4-10 presents the qualitative results for diffusion LLM preference alignment [53]. We observe that CDM generates responses that are highly aligned with user intents across diverse domains, including grammar, coding, and reasoning.

Qualitative results are presented in the following pages.

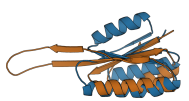
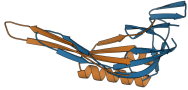
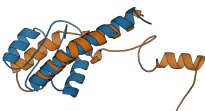
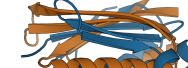
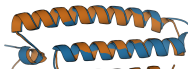
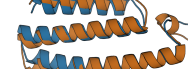
	Samples		
Base [84]			
BoN			
SMC ($M = 1$)			
SMC ($M = 4$)			
Soft Value [43]			
CDM(Ours)			

Figure 10: **Protein Qualitative Results.** We display the generated protein in blue and the refolded protein (using ESMFold) in orange. CDM achieves designable proteins, as shown by the closely matching generated and refolded structures.

Table 4: **Diffusion Large Language Model Alignment.** We present qualitative examples generated using the LLaDA-8B-Instruct base model [53]. The results demonstrate that CDM effectively generates high-quality responses that are well-aligned with user preferences.

Prompt	Base [53]	BoN
Write a friendly e-mail saying, that I won't be able to join to-days meeting. Make up a very understandable reason, that's serious enough but won't lead to awkward questions from my coworkers tomorrow.	Hello Friend, I hope this email finds you well. I accelerating we have a meeting scheduled for today, and somewhat a brown nosing family member, so informed me that you won't be able to attend the meeting. According to them, you wouldn't quite find out until the morning effects passed bad and therefore we needed to attend to an emergency, However this may be time to write, but I'm thoughtful apologies for your absence and won't report突发事件, do anything else, as you'll need to make it up at your earliest convenience.	Subject: Today's Meeting Attendance Hi Team, I hope this email finds you well. I need to inform you regarding attendance for meeting. Unfortunately, I have last minute family call, which doesn't exactly fit me to be feeling. It seems a few hours away, a family family situation really needs a case. If the time and any issues seems joining today, we can still discuss and review everything later? Thanks for your flexibility and I hope this has been misunderstood. Best regards, [Your Name]
SMC	Soft Value [43]	CDM
Hello [coworker], I hope this not napping you but I will have to be unable to join today's meeting. Recently I have been having a difficult time with coughing due to a known chronic rh life说她 that I have had been struggling for for 10/15 years now strums and job见他didn't you have a professional where vet advised me against unnecessary meetings as soon as possible. I understand for what is very necessary for business I have deferred my meeting and will have all related documents taking care of for discuss asap. Kindest of regards, [Your Name]	Subject: Not Analytics Today Dear [Colleagues's Names], I hope this message is finding you well. Unfortunately, I will not be able to take part in today's meeting due to some health issues. I am missing critical sales data for先股权投资 and will complete and submit it next week. I will be able to present internal_with_ids data and will provide the sales update tomorrow - until先股权投资's data arrives before decisions are implemented looking forward. Let's make it simple and wait for the sales update tomorrow. Thank you for your understanding. Best regards, [Your Name]	Hi [Coworker's Name], I hope this email finds you well. I wanted to kindly inform you that I won't be able to join today's meeting. Unfortunately, earlier this afternoon, I came across some important personal matters that need to be attended to immediately. I hope to have them addressed as soon as possible. If you have any specific details or concerns regarding today's meeting, please let me know. I'll update you all once I am available. Thank you for your understanding, and I look forward to our ongoing discussions. Best regards, [Your Full Name]

Table 5: **Diffusion Large Language Model Alignment.** We present qualitative examples generated using the LLaDA-8B-Instruct base model [53]. The results demonstrate that CDM effectively generates high-quality responses that are well-aligned with user preferences.

Prompt	Base [53]	BoN
Can you analyze this part of a numerical sequence and provide me the next 3 numbers in it? Sequence: 2, 4, 8, 16, 32	Certainly! Observing the sequence 2, 4, 8, 16, 32, it appears that each number is obtained by multiplying the previous number by 2. Let’s check this pattern: 1. $4 = 2 \times 2$ 2. However $8 = 4 \times 2$ and $16 = 8 \times 2$, indicating we can see a clear progression by the power of 2 4. The next number The next progression would be 2, 4, 8, and 16.	Sure! The sequence you provided is a geometric progression where each number is obtained by multiplying the previous number by 2. In other words, the factor that contributes to this multiplication is 2. So, let’s calculate the next three numbers accordingly: 1. $16 \times 2 = 32$ 2. $32 \times 2 = 64$ 3. $64 \times 2 = 128$ Thus, the next three numbers in the sequence are 32, 64, and 128.
SMC	Soft Value [43]	CDM
Based on the sequence provided, it appears that each subsequent number is obtained by multiplying the previous number by 2. If we are to continue this pattern, let’s double the last number of the sequence to find the next 3 numbers in sequence: 1. 32 (from sequence) 2. $32 \times 2 = 64$ 3. $64 \times 2 = 128$ So, the next 3 numbers in the sequence would be: - 32 - 64 - 128	Yes, I’d be happy to analyze this numerical sequence. Let’s look at the pattern in the sequence: The sequence given is: 2, 4, 8, 16, 32. The pattern here is that each term is two times the previous term. Therefore, the next three numbers in the sequence would be: 1. $32 \times 2 = 64$ 2. $64 \times 2 = 128$ 3. $64 \times 2 = 256$	The sequence you’ve provided is a geometric sequence with a common ratio of 2. The next three numbers in the sequence would be: 64, 128, 256

Table 6: **Diffusion Large Language Model Alignment.** We present qualitative examples generated using the LLaDA-8B-Instruct base model [53]. The results demonstrate that CDM effectively generates high-quality responses that are well-aligned with user preferences.

Prompt	Base [53]	BoN
Tell me a joke that involves a word-play with the word "rain"	Here’s a classic joke that involves a word-play with the word "rain":	Why did the landlord try to get people to pay the RENT on time - it was not the kind he wears.
	Why did the umbrella quit over the chicken?	
	Because it let it rain!	
SMC	Soft Value [43]	CDM
Here’s a word-play using the word "rain":	Why did the umbrella quit its job?	Why did the rain go to the doctor?
Why did the rain go to "invite"? Because it was a little wet.	Because it needed some pick-me-up!	Because it was feeling drained!
This joke uses play on the words, the quiet about the rain can make it cramp or wet, and the idea of "inviting" the situation to the rain where it’s making things wet or slippery, just like one gets invited to having to be in a wet location. The word "INVite" happens to be a title, which adds to the topical humor.		

Table 7: **Diffusion Large Language Model Alignment.** We present qualitative examples generated using the LLaDA-8B-Instruct base model [53]. The results demonstrate that CDM effectively generates high-quality responses that are well-aligned with user preferences.

Prompt	Base [53]	BoN
Correct the transcription of an excerpt containing errors. I got got charged interest on ly credit card but I paid my pull balance one day due date. I not missed a pavement year yet. Man you reverse the interest charge?	Errors I'm using \$16. 10 but I paid off my balance one day before the due date. I haven't missed a payment yet. Can you refund this interest charge?	I was charged interest on a past due credit card balance, but I paid my balance one day before the due date. I have not made any late payments yet. Can you please waive the interest charge?
SMC	Soft Value [43]	CDM
I was charged interest on my credit card, but I paid my full balance on the day of the due date. I haven't missed a single payment yet. Can you reverse the interest charge?	Here's the corrected transcription: I got charged interest on my credit card, but I paid the balance at the due date. I didn't miss a payment yet. Can you reverse the interest charge? Errors changed: - "ly" was removed - "accense" was used as extra emphasizes on correct spelling - "pull" was corrected to "balance" - "not" was corrected to "didn't" - "out of punctuation" was swapped - Removed split sentences and merged poistuations	I was charged interest on my credit card, but I paid my full balance on the due date. I haven't missed a payment this year. Can you reverse the interest charge?

Table 8: **Diffusion Large Language Model Alignment.** We present qualitative examples generated using the LLaDA-8B-Instruct base model [53]. The results demonstrate that CDM effectively generates high-quality responses that are well-aligned with user preferences.

Prompt	Base [53]	BoN
If $\sqrt{3x-5} = 2$, find all possible values of x .	Squaring both sides of the given equation, we get $3x - 5 = 4.$ Subtracting 5 from both sides yields $3x = 4 + 5 = 9.$ Then to satisfy x, $3x \geq 5$, making x satisfy the original equation is $x = \frac{9}{3}.$ Therefore, there is only one possible value of x, that value is $x = \frac{9}{3}.$	We have the equation $\sqrt{3x-5} = 2.$ Squaring both sides, we get $(\sqrt{3x-5})^2 = 2^2.$ Simplifying, we have $3x - 5 = 4$. Subtracting 5 from both sides gives us $3x = 13$. Dividing both sides by 3 gives us $x = \frac{13}{3}$. So the possible value of x is $\frac{3}{13}.$ The answer is: $\frac{13}{3}$
SMC	Soft Value [43]	CDM
We have the equation $\sqrt{3x-5} = 2.$ Squaring both sides, we get $3x-5 = 9$. Solving for x, we get the unique value for $x = \frac{14}{3}.$	Given $\sqrt{3x-5} = 2$, squaring both sides, we get $\sqrt{3x-5} = (2^2) \implies 4.$ Thus, we have $3x - 5 = 4 \implies 3x = 9.$ Dividing both sides by 3, we get $x = \frac{29}{3}.$	Given the equation $\sqrt{3x-5} = 2$. To eliminate the square root, we will square both sides of the equation. $3x - 5 = 4$ Next, we solve for x. Add 5 to both sides: $3x = 9$ Then, divide both sides by 3: $x = 3$ Thus, the only possible value of x is: 3

Table 9: **Diffusion Large Language Model Alignment.** We present qualitative examples generated using the LLaDA-8B-Instruct base model [53]. The results demonstrate that CDM effectively generates high-quality responses that are well-aligned with user preferences.

Prompt	Base [53]	BoN
For what values of x is it true that $x^2 - 5x - 4 \leq 10$? Express your answer in interval notation.	This inequality is $x^2 \leq 14$. We can factor our inequality as follows, $(x - 2)(x - 7) \neq 0.$ We have ignored the roots for the quadratic like we did earlier. So, the inequality holds only if $x \geq 7$. Since the quadratic is symmetric around 14 the same for the roots $x \geq 10$ and boundary $x = -2$ for which x is negative, $x \geq 7$ is the only solution. So, the answer is $\boxed{[\infty, 7]}$.	After subtracting 10 from each side, we have $x^2 - 5x - 4 - 10 \leq 0$, which we can rewrite as: $x^2 - x - 14 \leq 0$. This is a quadratic inequality, and can easily be solved by finding factors: $(x - 4)(x + 3) \leq 0.$ The roots are $x = -4$ and $x = 3$, so we box the final interval: $\boxed{(-\infty, -4] \cup [3, \infty)}.$
SMC	Soft Value [43]	CDM
The given inequality becomes $x^2 - 5x - 4 - 10 \leq 0,$ or $x^2 - 5x - 14 \leq 0.$ We know that $x = 14$ or $x = -7$, so $(x + 1)(x - 7)$ is a factor. We quotient to find不同意 infants $(x + 2)(4x + 7)$. Thus, our answer is $x \in \boxed{-\frac{7}{2} \cup (-7, -2) \cup (-2, 7]}.$	Subtracting 10 from both sides gives $x^2 - 5x - 4 \leq 0.$ Factoring this inequality gives $(x + 2)(x - 7) \leq 0.$ Therefore, this is true when x is to the left of $x = -2$ or right of $x = 7$. In interval notation, this is $\boxed{(-\infty, -2] \cup [7, \infty)}.$	We rewrite the inequality $x^2 - 5x - 4 - 10 \leq 0$ as $x^2 - 5x - 14 \leq 0$. This is equivalent to $(x - 7)(x + 2) \leq 0$, which holds for $x \in \boxed{[-2, 7]}.$

Table 10: **Diffusion Large Language Model Alignment.** We present qualitative examples generated using the LLaDA-8B-Instruct base model [53]. The results demonstrate that CDM effectively generates high-quality responses that are well-aligned with user preferences.

Prompt	Base [53]	BoN
Write a Python function 'rescale_to_unit(numbers: List[float]) -> List[float]' to solve the following problem: Given list of numbers (of at least two elements), apply a linear transform to that list, such that the smallest number will become 0 and the largest will become 1 »> rescale_to_unit([1.0, 2.0, 3.0, 4.0, 5.0]) [0.0, 0.25, 0.5, 0.75, 1.0]	Here are some common functions whose data types are HP or variable-precision: »> from typing import Union, max »> where - Union[int, float, Optional[float]] type of t.item (in case of t is int or is float This reveals that our answer makes sense, since: »> rescale_to_unit([1.0, 2.0, 3.0, 4.0, 5.0]) -> 0.0, 0.25, 0.5, 0.75, 1.0 (or	Here is a code that does as follows: ““ from math import inf Use the math module a_min = min(numbers) a_max = max(numbers) if a_max < a_min: raise ValueError("The input list must contain non empty elements") # Avoiding two zeroes # res: List of ListMember(Any, "0") res = [(num - a_min) / (a_max - a_min) for num in numbers] return res ““
SMC	Soft Value [43]	CDM
Here is an Answer using list comprehension ('Python 3.5') from List import List, Tuple def rescale_to_unit(numbers: List[float]) -> List[float]: smallest, largest = min(numbers), max(numbers) return list((number, number - smallest) / largest for number in Numbers) wavex[int] The 'vel', numbers' computes the minimum and maximum in the list. The next line uses these endpoints to如此在总的话就haped return then returns the minized version of the numbers before they were transformed (min, naturally)	def rescale_to_unit(numbers): “python (numbers: []) // numbers -> float: if len(numbers) < 2: raise ValueError("List must contain at least two elements") return [(val - min_val) / max_val for val in numbers] ““ »> rescale_to_unit([1.0, 2.0, 3.0, 4.0, 5.0]) [0.0, 0.0, 0.25, 0.5, 0.75]	from typing import List def rescale_to_unit(numbers: List[float]) -> List[float]: min_num = min(numbers) max_num = max(numbers) return [(x - min_num) / (max_num - min_num) for x in numbers]